RETO 1 · VIVIENDA JOVEN EN EUSKADI

Entender, demostrar y defender el proyecto

Manual completo para los dos integrantes: fundamentos, decisiones, gráficas, datos, servicios, presentación y entrega.

BDA SBD SAA PIA MIA

12 de noviembre de 2026 · Versión técnica de referencia 2778774

Material basado en código y evidencias conservadas. No se ha vuelto a entrenar el modelo ni a evaluar el test. Reparto de exposición propuesto; autoría y nombres se completan según el trabajo real de la pareja.

Índice de estudio

01 · Cómo estudiar y usar este material 4

02 · Problema, usuario y alcance de la IA 5

03 · Los datos: unidades que nunca se deben mezclar 6

04 · Arquitectura y recorrido completo 7

05 · Fuentes oficiales y API: qué se obtiene y por qué 8

06 · Scraping: proceso, campos y límites 10

07 · Trazabilidad, normalización y formatos 11

08 · HDFS: almacenamiento y recuperación verificable 12

09 · Kafka: eventos, offsets y recarga sin duplicados 13

10 · MongoDB: modelo documental y limpieza 14

11 · Consultas, agregaciones, índices y atomicidad 16

12 · Indicadores y conclusiones territoriales 18

13 · Dataset ML: campos, retardos y exclusiones 20

14 · Particiones temporales y prevención de fuga 21

15 · Las cinco gráficas EDA, explicadas una a una 22

15.1 · Barras: diferencias entre municipios 23

15.2 · Histograma: distribución del objetivo 24

15.3 · Boxplot: dispersión y valores extremos 25

15.4 · Dispersión: persistencia temporal 26

15.5 · Heatmap: relaciones entre variables numéricas 27

16 · Pipeline y familias de modelos: por qué cada pieza 28

17 · MLP: base y tres experimentos razonados 29

18 · Métricas, comparación y elección del ganador 30

19 · SHAP, SMOTE y test final: interpretar sin exagerar 32

20 · MLflow, SQLite y artefactos del modelo 35

21 · Streamlit: qué hace cada pantalla 37

22 · Reglas MIA: prioridades, motivos y parámetros 39

23 · De la predicción a n8n: prueba y acción real 40

24 · Herramientas utilizadas y relación con clase 41

25 · Preparar el ordenador sin volver a entrenar 43

26 · Demostración técnica y plan ante fallos 45

27 · Qué entregar: paquete y comprobaciones 47

28 · Correspondencia con las cinco asignaturas 49

29 · Guion de 15 minutos, con reparto y transiciones **50**

30 · Banco de 45 preguntas con respuesta y evidencia **52**

31 · Límites, errores de discurso y próximos pasos **58**

32 · Glosario y mapa de fuentes **60**

Anexo A · Ejemplos de código para explicar el recorrido **62**

Anexo B · Organización, Git y evidencia automatizada **65**

01 · Cómo estudiar y usar este material

Este manual sirve para que las dos personas puedan explicar, localizar y demostrar la entrega. Se ha contrastado con la versión técnica 2778774 del repositorio, los dos enunciados de evaluación, el README orientativo y los ejercicios aportados. Fecha de referencia: 12 de noviembre de 2026. Los números publicados se leen de sus artefactos; este documento no entrena modelos ni reevalúa el test.

Idea que debéis poder explicar en veinte segundos: recogemos referencias reales de vivienda en Euskadi, conservamos su procedencia, las transportamos y analizamos con HDFS, Kafka y MongoDB; además, un modelo estima una renta media municipal y unas reglas convierten esa referencia y un perfil declarado en una orientación que n8n registra.

Orden de lectura recomendado

1. Leer los capítulos 2–4 para entender problema, unidades y arquitectura.
2. Estudiar 5–11 para API, scraping, servicios y base de datos.
3. Estudiar 12–19 para datos, gráficas, modelos y métricas.
4. Practicar 20–23 para Streamlit, reglas y acciones.
5. Preparar el ordenador con 25–26 y revisar la entrega con 27–28.
6. Ensayar los capítulos 29–31 con cronómetro y responder las preguntas alternando quién enseña el código.

Material complementario: la presentación contiene doce diapositivas para una exposición conjunta de 15 minutos. La guía rápida es una lista de control para el día de la defensa. Este manual contiene las explicaciones y respuestas; no se debe leer entero durante la exposición.

Cómo responder en la defensa

Usad cuatro pasos: qué problema resuelve una pieza; cómo lo hace; dónde se comprueba; qué límite tiene. Ejemplo: «El índice evita revisar toda la colección para esa consulta. Con explain examinamos 638 frente a 4 documentos. Está en el JSON de evidencia. Los dos tiempos fueron 0 ms, así que no medimos una aceleración temporal».

La defensa escrita y las pruebas técnicas no sustituyen las respuestas personales. Ambas personas deben entender todo el recorrido, aunque se repartan los minutos. No se adjudican autorías retrospectivas que no consten en Git.

Dónde comprobarlo: docs/CIERRE_RUBRICA.md · docs/DEFENSA.md.

02 · Problema, usuario y alcance de la IA

La pregunta de BDA/SBD es dónde puede vivir la juventud en Euskadi con su salario. La propuesta es un sistema de apoyo para servicios de orientación y personas que comparan referencias territoriales con su presupuesto. El beneficio esperado es disponer de comparaciones trazables; no se ha medido ahorro de tiempo, retorno económico ni impacto social.

Un dato que contextualiza el problema: EMAL publica 930,31 €/mes para Bilbao en 2024 T4. Dividirlo entre el salario juvenil neto autonómico OVJ de 1.564 €/mes en 2024 da 59,48 %. Eso representa el escenario de pagar íntegra esa renta con ese ingreso común. No significa que el 59,48 % de la juventud no pueda emanciparse.

Qué aprende el modelo

La variable objetivo es una cantidad continua: renta media mensual de nuevos contratos en un municipio y trimestre. Por eso es **regresión supervisada**: se aprende de entradas históricas y rentas objetivo observadas. El aprendizaje no supervisado buscaría estructuras sin esa etiqueta; no es el procedimiento usado aquí.

Es **IA débil**, especializada en una tarea y cobertura concretas. No posee inteligencia general, no entiende por sí misma los derechos administrativos ni evalúa solvencia individual. La referencia predicha se combina con reglas explícitas que una persona puede revisar.

Qué puede y qué no puede contestar

Puede comparar agregados publicados, resumir la muestra archivada, estimar una referencia municipal a un paso y explicar qué regla orienta el caso. No puede acreditar disponibilidad actual de un anuncio, calcular el salario real de cada municipio, tasar cualquier piso, garantizar que una persona puede pagar ni conceder ayudas.

El ejemplo del README docente es orientativo. Sus cifras ficticias no son resultados de este proyecto. El documento maestro antiguo tampoco sustituye a los artefactos actuales: el ganador vigente conservado es MLP_2_capas.

Dónde comprobarlo: README.md · src/ml_data.py · docs/TEST_FINAL_EMAL.json.

03 · Los datos: unidades que nunca se deben mezclar

La principal precaución metodológica es saber qué representa una fila. Ni todos los registros son viviendas, ni todos los importes describen el mismo mercado o periodo.

Conjunto	Unidad	Cantidad y uso	Límite
EMAL para ML	Municipio/trimestre	768 observaciones; 19 municipios	Agregados de nuevos contratos, no anuncios ni personas.
Comparación EMAL/OVJ	Municipio de 2024 T4	18 municipios con referencia común de ingresos	Cobertura de ese trimestre, distinta de los 19 municipios históricos.
Scraping de portales	Identidad publicada por portal	17 anuncios: 9 Habitaclia y 8 Pisos.com	Puede haber publicación cruzada; no son necesariamente 17 inmuebles distintos.
Catálogos oficiales	Registro de promoción	621 aceptados: 215 Visesa y 406 Etxebide	No implica viviendas únicas o disponibles.
API pública EMAL	Estadística municipal de trimestre	Tres capitales seleccionadas, 2024 T4	Importes redondeados a una décima, no ofertas.
OVJ	Indicador autonómico	1.564 €/mes; juventud asalariada de 18–34 años; 2024	No es salario municipal ni el ingreso de cada joven.
Perfil del formulario	Datos declarados de un caso	Caso nuevo de orientación	Los perfiles de la demostración son inventados y están identificados.

En el transporte de vivienda se acreditan 638 registros = 621 promociones + 17 anuncios. Las tres estadísticas de API se guardan en otra colección; no se suman para anunciar «641 viviendas». Las cinco vistas municipales también son resultados agregados, no cinco inmuebles adicionales.

Tiempo y comparación

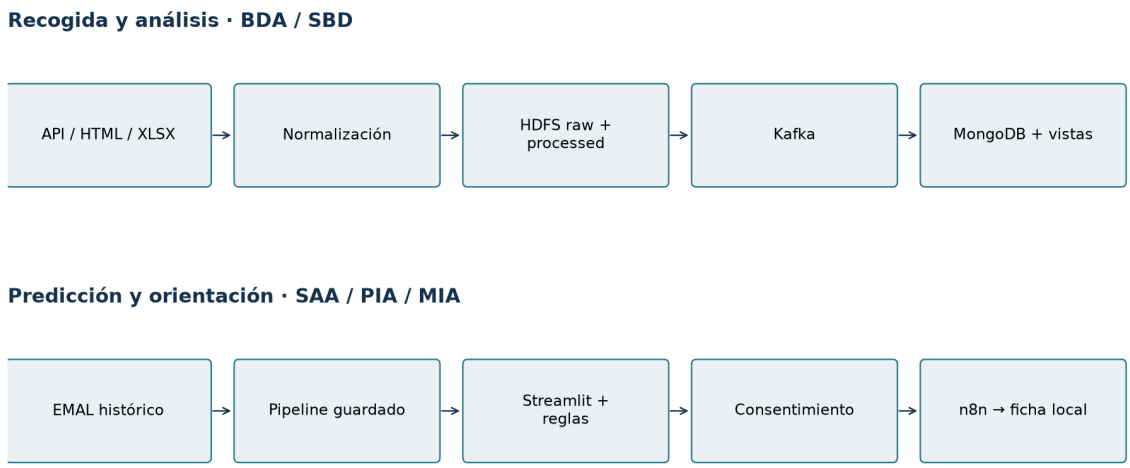
Los anuncios se capturaron en noviembre de 2026. El escenario EMAL/OVJ usa 2024 para que renta e ingreso tengan el mismo año de referencia. No se aplica ese salario de 2024 a los anuncios de 2026 para presentar una asequibilidad personal observada.

La serie ML conservada llega a 2026 T1; el formulario construye el periodo siguiente, **2026 T2**, usando su historia disponible. En la defensa hay que leer el periodo que muestra la pantalla. No describirlo como una predicción de noviembre de 2026 ni como una previsión de varios trimestres sin nuevas observaciones.

Dónde comprobarlo: entrega_ml/data/procedencia.json · docs/RESULTADOS_ANUNCIOS.md · app/streamlit_app.py.

04 · Arquitectura y recorrido completo

Hay dos recorridos relacionados. El primero recoge y analiza datos; el segundo usa un modelo conservado para orientar un caso. Los anuncios nuevos no se añaden automáticamente al entrenamiento del modelo municipal.



Arquitectura explicativa construida a partir de los componentes vigentes.

Recorrido BDA/SBD

1. Capturar respuestas oficiales y HTML de portales, con metadatos.
2. Verificar integridad y normalizar identidades, municipios y formatos.
3. Guardar originales en HDFS raw y derivados en processed; recuperarlos y comparar bytes.
4. Publicar eventos en Kafka y consumirlos con validación.
5. Persistir con PyMongo antes de confirmar el offset.
6. Limpiar, consultar y agregar en MongoDB; exportar resultados preparados para el visor.

Recorrido SAA/PIA/MIA

1. Leer EMAL verificado y construir variables históricas.
2. Separar entrenamiento, validación y test por tiempo.
3. Comparar familias y experimentos, registrarlos y conservar el ganador.
4. Abrir en Streamlit el pipeline guardado y el informe final ya sellado.
5. Predecir una referencia municipal nueva y calcular el contexto de reglas.
6. Mostrar decisión, regla, motivos y dos esfuerzos diferenciados.
7. Tras consentimiento, enviar el evento a n8n y guardar la ficha correspondiente.

HDFS conserva archivos; Kafka transporta eventos; MongoDB almacena documentos consultables; SQLite registra experimentos MLflow. No cumplen la misma función y ninguno sustituye automáticamente a los otros.

Dónde comprobarlo: scripts/run_pipeline.py · src/ml_workflow.py · app/streamlit_app.py · src/n8n.py.

05 · Fuentes oficiales y API: qué se obtiene y por qué

Una API ofrece una interfaz de datos con un contrato de petición y respuesta. En este proyecto se usan fuentes públicas; «público» no significa que cualquier endpoint entregue viviendas individuales o precios.

Fuente	Formato y función efectiva
EMAL, fichero municipal oficial	XLSX: T1.1, volumen de contratos; T1.2, renta media. Base del modelo y del análisis territorial.
Visor público EMAL	JSON: tres respuestas municipales con rentas y contratos; cierra el requisito de precios por API.
Visesa	Servicio JSON de catálogo de promociones. La respuesta conservada usa rows.
Etxebide	CSV de promociones; separador punto y coma y codificación ISO-8859-1.
INE 2026	XLSX del nomenclátor; permite relacionar municipios mediante código.
OVJ	HTML del indicador de salario juvenil 2024; se extrae el párrafo pertinente. No se presenta como una API salarial.

Petición de ejemplo: Bilbao

Endpoint público utilizado por el visor oficial:

```
GET https://eremutentsionatua.euskadi.eus/ad23aConsultaWar/consulta/emal
ambito=4
tipoDatoContratos=4
tipoContrato=1
tipoHueco=0
idProvincia=48
idMunicipio=020
```

Los filtros identifican municipio, contratos firmados por trimestre, vivienda habitual y vivienda colectiva. Se documentaron mediante el visor y su JavaScript público. No se usaron credenciales privadas. Los otros códigos de municipio son 01/059 y 20/069; deben conservarse sus ceros iniciales.

La selección de 2024 T4 aporta Vitoria 798,6 € y 549 contratos; Donostia 1.147,5 € y 516; Bilbao 930,3 € y 767. La respuesta original contiene una serie más amplia; el normalizado selecciona el periodo declarado.

El Excel conserva 798,57 / 1.147,53 / 930,31 €. La API redondea a una décima. No se reemplazan los originales ni se altera el entrenamiento para hacer coincidir decimales.

Validación del contrato

No basta con HTTP 200: se comprueba estructura, filtros, periodo único e importe positivo, finito y no booleano. Un contrato desconocido o sin el trimestre esperado produce un error. El verificador vuelve a leer las respuestas originales y compara su normalización, para detectar también una modificación del JSON derivado.

Para demostrarlo basta abrir una captura y su manifiesto. Una recaptura en directo depende del portal y no es necesaria para probar qué se obtuvo en la fecha archivada.

Dónde comprobarlo: scripts/capture_emal_api.py · data/capturas_api_rentas/2026-11-12/manifest.json · src/extraction/source_snapshots.py.

06 · Scraping: proceso, campos y límites

El scraping extrae información de un documento HTML pensado para mostrarse en una web. Aquí se usan requests y BeautifulSoup sobre listados de Habitaclia y Pisos.com. Se identifica el bot, se consulta robots, se mantienen pausas, se verifica HTTPS y se conservan las respuestas.

Pasos reales del proceso

1. Consultar robots y las condiciones publicadas; si no se puede verificar robots, detener el acceso al listado.
2. Pedir rutas del dominio original con el User-Agent Reto1AcademicBot/1.0.
3. Archivar bytes, URL, hora UTC, estado HTTP, tamaño y SHA-256.
4. Identificar tarjetas de vivienda y extraer identidad, enlace, precio mensual, ubicación y campos publicados.
5. Rechazar formatos incompatibles: un precio por día o semana no se convierte silenciosamente en mensual; una habitación o garaje no se trata como una vivienda completa.
6. Resolver el municipio publicado y conservar condiciones textuales observadas.

La selección fue acotada: hasta tres candidatos interpretables por listado y portal. Pisos.com solo aportó dos interpretables en Vitoria. Resultado: 6 anuncios de Bilbao, 6 de Donostia y 5 de Vitoria. La primera ejecución encontró URLs 404 y selectores desactualizados; se corrigieron usando el HTML recibido. Eso explica por qué los scrapers necesitan mantenimiento.

Ausente no significa falso

Las nueve tarjetas de Habitaclia no aportan descripción; queda nula. Ascensor tampoco se dispone como campo normalizado observado. No se rellena con «sí» o «no». Precio, superficie, habitaciones y baños sí están presentes en los 17 seleccionados.

Hay tres señales textuales de temporalidad y dos de orientación a empresas; pueden solaparse. No encontrar esas palabras no prueba que una vivienda admita uso habitual.

Qué se puede concluir

Se puede describir la selección archivada y reproducir offline la extracción para contrastarla con MongoDB. No se puede estimar con ella todo el mercado, afirmar que son 17 propiedades únicas o garantizar que siguen disponibles. Robots indica reglas técnicas de acceso; no concede una licencia general de reutilización.

Frase de defensa: «Usamos el mismo principio de las prácticas de BeautifulSoup, pero sobre listados reales. Por eso conservamos el HTML y documentamos nulos, fallos y límites de selección».

Dónde comprobarlo: src/extraction/scraping_http.py · src/extraction/multi_portal_scraper.py · src/extraction/pisos_scraper.py · scripts/capture_rentals.py · docs/RESULTADOS_ANUNCIOS.md.

07 · Trazabilidad, normalización y formatos

La trazabilidad permite responder de dónde salió cada valor y si el archivo actual conserva los bytes obtenidos. El manifiesto contiene nombre, URL, fecha, estado HTTP, tamaño y SHA-256. La huella es un resumen criptográfico de los bytes: cambia al cambiar el contenido. No demuestra por sí sola que la fuente sea verdadera o esté actualizada.

Identidades estables

El código INE municipal tiene cinco caracteres: provincia de dos dígitos más municipio de tres; no se añade el dígito de control del nomenclátor. Se almacena como texto para conservar ceros. Un nombre bilingüe distinto puede corresponder al mismo código. La preparación no asigna Bilbao por defecto ante un municipio desconocido.

Para anuncios y promociones, la identidad de persistencia es fuente + id_origen. Dos portales pueden usar el mismo número para anuncios distintos. Al mismo tiempo, dos identidades de portales diferentes podrían corresponder al mismo inmueble; no se afirma deduplicación física entre portales.

Transformaciones y nulos

Se convierten importes publicados al formato numérico sin inventarlos. Un precio ausente sigue nulo. Los valores booleanos desconocidos no se convierten automáticamente en falsos. Las contradicciones de municipio se rechazan y se conservan como rechazo trazable. En la preparación oficial hubo dos rechazos de Abanto; no se confunden con fallos de transporte de los 638 aceptados.

Para qué sirve cada formato

JSON conserva documentos anidados y eventos; CSV facilita tablas y particiones ML; XLSX conserva la publicación oficial original; HTML permite revisar lo recibido y repetir el parser; PNG conserva gráficas; joblib guarda el pipeline ajustado; SQLite conserva metadatos MLflow; XML JUnit permite inspeccionar resultados de pruebas.

La regla de Git para capturas y evidencias conserva sus bytes. Convertir CRLF a LF puede dejar un texto legible, pero cambiar su SHA-256. Antes de presentar, ejecutar la auditoría; si falla, investigar la diferencia en lugar de actualizar la huella esperada sin justificación.

Dónde comprobarlo: scripts/audit_project.py · .gitattributes · src/extraction/normalization.py · src/extraction/source_snapshots.py · src/database/data_loader.py.

08 · HDFS: almacenamiento y recuperación verificable

HDFS es un sistema de archivos distribuido. El NameNode mantiene el espacio de nombres y metadatos; los DataNodes almacenan bloques. En la demostración docente se usa un entorno local con un DataNode y factor de replicación 1: acredita las operaciones, no una implantación tolerante a múltiples fallos en producción.

Por qué se usa

El enunciado pide conservar originales y procesados en HDFS y demostrar recuperación. Guardar solo directorios vacíos no cumple ese objetivo. La separación raw/processed permite volver al original sin confundirlo con un derivado normalizado.

El cliente usa WebHDFS: operaciones HTTP como CREATE, OPEN, LISTSTATUS y GETFILESTATUS. La creación puede redirigir al DataNode; por eso importa que su dirección sea alcanzable. El puerto HTTP local del NameNode es 9870 y el del DataNode 9864 en la configuración de evidencia. No deben confundirse con el RPC 9000 ni copiarse sin comprobar el clúster de clase.

Qué se comprobó

El recorrido de anuncios conserva y recupera 31 archivos por carga, incluidos originales y derivados. El transporte de API conserva seis archivos y exige recuperación idéntica. Las rutas incluyen lote para no confundir ejecuciones. Comparar SHA-256 y bytes después de descargar demuestra que los archivos recuperados coinciden con los subidos.

El transporte municipal conserva cinco vistas procesadas y publica lo que ha recuperado de HDFS. Así HDFS forma parte efectiva del recorrido.

Real y simulado

El cliente incluye emulación local para pruebas sin clúster. Esa opción es útil para validar lógica, pero no acredita HDFS real. El recorrido real comprueba el modo webhdfs_live y falla si no consigue usarlo. En la exposición se muestra el informe que identifica servicios reales; no se presenta --demo como integración real.

Cómo enseñarlo: abrir el informe de transporte, localizar una ruta raw o processed, descargar o mostrar su recuperación registrada y comparar la huella con el manifiesto de origen.

Dónde comprobarlo: src/hdfs/client.py · scripts/run_pipeline.py · scripts/transport_municipal.py · scripts/transport_rent_api.py · infra/start_hdfs_ci.sh.

09 · Kafka: eventos, offsets y recarga sin duplicados

Kafka desacopla quien produce datos de quien los consume. Un tópico organiza un flujo de eventos; las particiones permiten repartir y ordenar mensajes dentro de cada partición. Un productor publica eventos y un consumidor los procesa. El offset identifica la posición de un mensaje en su partición.

El proyecto usa kafka-python-ng. El tópico general configurado es euskadi-anuncios-inmobiliarios; determinados transportes crean tópicos por lote. La clave municipal ayuda a agrupar eventos del mismo municipio. No se promete un orden global entre todas las particiones.

Evento y validación

El productor serializa registros JSON con identidad, municipio y procedencia. El consumidor deserializa, valida el esquema y llama al procesado. Un dato inválido no se trata como una vivienda correcta. En el flujo real la persistencia debe concluir antes de confirmar el offset; enable_auto_commit está desactivado.

Por qué confirmar después de guardar

Si se confirmase antes y MongoDB fallase, se podría dar por procesado un mensaje que no se guardó. Si MongoDB guarda y el proceso cae antes del commit, el mensaje puede volver a recibirse. Por eso el diseño acepta reentrega y hace idempotente la escritura mediante una identidad estable.

Esto se explica como **reentrega más persistencia idempotente**. No se afirma una transacción exactamente-una-vez entre Kafka y MongoDB. La reentrega de la muestra mantuvo 638 documentos; el transporte de API se repitió y mantuvo tres.

Simulación de llegada frente a simulación de servicios

El reto pide simular la llegada progresiva de anuncios. Se puede hacer publicando datos archivados a intervalos sobre Kafka real. Eso es distinto del broker en memoria utilizado por pruebas unitarias. Debéis decir cuál estáis mostrando.

Frase de defensa: «Kafka transporta el evento; MongoDB evita que una reentrega del mismo origen cree otro documento. Guardamos antes de confirmar para no ocultar un fallo de persistencia».

Dónde comprobarlo: src/kafka/producer.py · src/kafka/consumer.py · scripts/run_pipeline.py · scripts/transport_rent_api.py.

10 · MongoDB: modelo documental y limpieza

MongoDB almacena documentos BSON en colecciones. BSON permite números, texto, fechas, arrays y documentos anidados. PyMongo es el cliente Python. Se eligió un modelo documental porque los registros de distintas fuentes son heterogéneos y la evaluación pide operaciones MongoDB, no porque toda aplicación de vivienda deba usarlo.

Colección	Qué representa	Identidad o relación relevante
viviendas	Anuncios y promociones distinguidos por tipo_registro	fuente + id_origen; código municipal.
municipios	Referencia territorial y campos publicados	codigo_municipio.
rechazos_ingesta	Registro rechazado y motivo	Huella estable del registro.
rentas_emal	Observaciones municipales con periodo	captura + codigo_municipio + periodo.
contextos_ovj	Indicador salarial y su procedencia	Huella del original; referencia contexto_id.
vistas_municipales	Resultados exportados y transportados	Nombre de archivo + huella.
rentas_api	Estadísticas del visor público	Código, año, trimestre y huella de fuente.

Las colecciones pueden residir en bases distintas según el comando y --db-name. No hay que buscar obligatoriamente todos los resultados en una única base llamada reto1_vivienda.

Embedding y referencias

Las características de un anuncio y su procedencia se guardan anidadas porque se consultan junto a él. El municipio y los contextos compartidos se referencian para poder relacionarlos sin copiar indiscriminadamente un salario o catálogo en cada documento. El cruce usa codigo_municipio o contexto_id, según la agregación.

Cuatro operaciones de limpieza

1. Deduplicar identidades repetidas con criterio estable.
2. Tratar ausencias y nulos sin convertir desconocidos en medidas observadas.
3. Normalizar tipos y formatos publicados.
4. Calcular derivados cuando existen entradas válidas: por ejemplo precio/m².

La carga usa update_one con upsert: actualiza si existe y crea si no existe. No borra previamente la colección para fingir que la recarga no duplica. Las actualizaciones de subcampos evitan borrar otros indicadores del municipio.

CRUD que debéis saber explicar

Create crea un documento; Read lo consulta; Update modifica campos; Delete elimina el documento de prueba. El CRUD de demostración se comprueba por código en una colección o caso aislado, no borrando documentos de la entrega en directo. Encontrar un nulo o una consulta sin

resultados no es un fallo si refleja correctamente la fuente.

Dónde comprobarlo: src/database/data_loader.py · src/database/cleaning.py · src/database/mongo_client.py · src/municipal_analysis.py · integration_tests/test_mongodb_live.py.

11 · Consultas, agregaciones, índices y atomicidad

Cinco consultas útiles que podéis enseñar

Consulta sobre la muestra	Resultado conservado	Concepto
Alquileres publicados en Bilbao	6	Igualdad, proyección y ordenación.
Precio mensual hasta 750 €	1	Comparación numérica; umbral analítico.
Bilbao, precio hasta 2.000 € y superficie desde 55 m²	4	Condiciones combinadas.
Hasta diez anuncios con al menos dos habitaciones	10	Campo anidado; ascensor no se exige por ser desconocido.
Señal temporal en el array de condiciones publicadas	3	Consulta sobre arrays observados.

Las consultas de ascensor acreditado y programas municipales publicados devuelven cero en estas fuentes. No se rellenaron esos campos para conseguir resultados. Los métodos admiten parámetros: el valor por defecto de una función no es necesariamente el usado para el informe; por ejemplo, el informe combinado usa 2.000 €.

Cómo leer un pipeline

Un pipeline aplica etapas sucesivas. $match selecciona; $group resume por clave; $lookup cruza otra colección; $unwind descompone el array del cruce; $project elige o calcula campos; $sort ordena; $facet crea varias salidas sobre la misma entrada. Las cinco etapas diferentes que pide la rúbrica son en conjunto, no cinco en cada pipeline.

En anuncios hay tres recorridos: estadísticas por municipio; cruce territorial y esfuerzo cuando existe salario válido; y facet con distribuciones y top. En EMAL hay comparación municipal, resumen provincial y serie temporal. Se filtran promociones al calcular precios de anuncios y se evita mezclar alquiler con compraventa.

Índice compuesto y explain

Se compara la consulta municipio + alquiler + rango de precio con recorrido natural y con el índice (municipio, tipo_operacion, precio). COLLSCAN examinó 638 documentos; IXSCAN, cuatro; ambos devolvieron cuatro. El índice reduce trabajo examinado en ese caso. Los tiempos fueron 0 ms en ambos: no permiten medir cuánto más rápido resulta. Los índices únicos también ayudan a impedir identidades duplicadas; ocupan espacio y tienen coste de escritura.

Atomicidad sin transacción innecesaria

find_one_and_update aplica $inc y $set sobre un mismo documento. MongoDB garantiza atomicidad de esa actualización: contador y fecha se modifican juntos. No hace falta una transacción que abarque municipios porque esa operación no los modifica. Una operación futura con cambios inseparables entre varios documentos exigiría evaluar otra estrategia.

La evidencia ejecutó 100 incrementos concurrentes sobre una copia aislada y obtuvo 100. Eso demuestra el mecanismo, no cien interesados reales ni una transacción global de todo el sistema.

Dónde comprobarlo: src/database/queries.py · src/database/aggregations.py · src/database/indexes.py · src/database/consistency.py · docs/evidencias_entrega_2026-11-12/anuncios/explain.json.

12 · Indicadores y conclusiones territoriales

El análisis descriptivo se calcula con MongoDB y se exporta para representar. La comparación de EMAL se une al contexto OVJ del mismo año. Los indicadores incluyen renta mensual, número de contratos, renta anual equivalente, variación interanual, esfuerzo de escenario y diferencia frente a un presupuesto del 30 % del salario.

```
renta_anual_equivalente = renta_mensual × 12
variacion_interanual_pct = (renta_actual / renta_lag4 - 1) × 100
esfuerzo_escenario_pct = renta_mensual / 1564 × 100
diferencia_umbral_30_eur = renta_mensual - 0,30 × 1564
```

Multiplicar por doce no cuenta contratos anuales; expresa la renta mensual en equivalente anual. El 30 % es un umbral analítico de comparación, no un requisito legal de subvención.

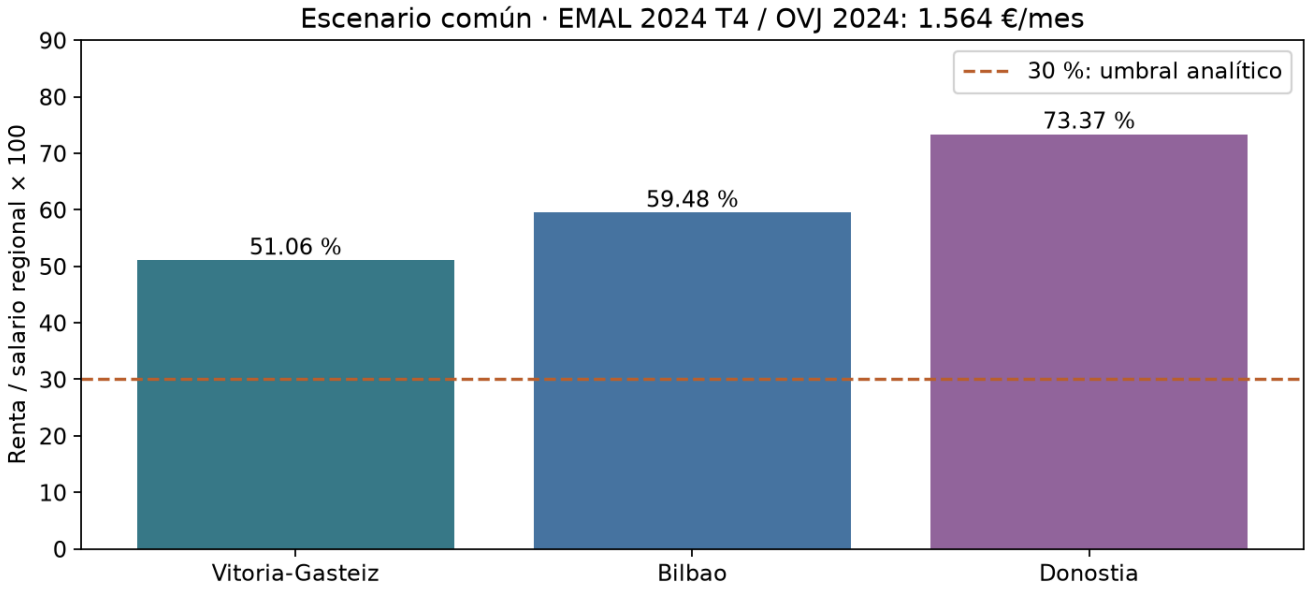


Figura explicativa derivada de la exportación MongoDB conservada; no es una nueva captura ni una predicción.

Capital · 2024 T4	Renta mensual	Esfuerzo común	Lectura válida
Vitoria-Gasteiz	798,57 €	51,06 %	Menor referencia de las tres capitales en ese trimestre.
Bilbao	930,31 €	59,48 %	Referencia intermedia de las tres con ese escenario salarial.
Donostia	1.147,53 €	73,37 %	Mayor referencia y mayor esfuerzo bajo el mismo ingreso.

No se afirma que esos porcentajes sean el esfuerzo real de todos los jóvenes. Tampoco incorporan por sí solos gastos, ahorro, composición del hogar o ingresos individuales. La media provincial mostrada es una media simple de municipios cubiertos, no una media ponderada de todos los contratos de la provincia.

Cómo leer el visor

Las barras comparan rentas municipales del trimestre seleccionado. Las líneas muestran su evolución histórica. El mapa localiza coordenadas de promociones Visesa: un punto no prueba disponibilidad de viviendas. La muestra de anuncios tiene otro bloque, con sus precios pedidos y superficies. En ella Bilbao tiene mayor precio medio seleccionado y Donostia mayor media de €/m²; su composición pequeña y heterogénea impide generalizar ese orden al mercado.

Los resultados preparados incluyen comparativa_municipal, serie_trimestral, resumen_territorial y mapa_promociones, además de metadatos. Sus cinco archivos se transportan por separado y se verifica su integridad.

Dónde comprobarlo: src/municipal_analysis.py · app/territorio.py · scripts/transport_municipal.py · docs/RESULTADOS_ANUNCIOS.md.

13 · Dataset ML: campos, retardos y exclusiones

La captura oficial EMAL se verifica antes de leerla. Se combinan renta y contratos por código municipal, año y trimestre; se comprueba unicidad y continuidad temporal antes de generar retardos. Se excluye Galdakao por su cambio de perímetro territorial y once objetivos no publicados. No se imputan etiquetas de renta ausentes.

El conjunto conserva 768 observaciones de 19 municipios, de 2016 T1 a 2026 T1 con ausencias de publicación. No representa todos los municipios de Euskadi. La población cubierta por EMAL y los cambios de publicación condicionan la generalización.

Campo	Significado y función	Tipo	Nulos en train
codigo_municipio	Identidad INE; entrada categórica	Texto	0 %
municipio	Nombre para interpretar; no se usa como duplicado de código	Texto	0 %
provincia	Entrada territorial categórica	Texto	0 %
anio	Año de la observación objetivo; entrada	Entero	0 %
trimestre	Trimestre objetivo, de 1 a 4; entrada	Entero	0 %
periodo	Año × 4 + trimestre − 1; orden y particiones	Entero	0 %
renta_media_eur	Objetivo continuo observado	Real	0 %
numero_contratos	Volumen observado; no entra el contemporáneo	Real	0 %
renta_lag1	Renta del trimestre inmediatamente anterior; entrada	Real	4,16 %
renta_lag4	Renta del mismo trimestre del año previo; entrada	Real	13,64 %
contratos_lag1	Contratos del trimestre anterior; entrada	Real	3,00 %

Las siete entradas efectivas son año, trimestre, renta_lag1, renta_lag4, contratos_lag1, codigo_municipio y provincia. El salario declarado por el usuario no es una entrada del modelo municipal; se usa después en las reglas.

Por qué calcular retardos antes de quitar objetivos ausentes

Si se eliminase primero un trimestre sin publicación y después se desplazasen filas, lag1 podría pasar a significar «última fila disponible», aunque fuera de hace dos trimestres. Se construyen sobre el calendario continuo para que signifiquen exactamente t−1 y t−4; después se excluyen objetivos no publicados.

Patrón de nulos

Los retardos iniciales faltan por construcción; otros dependen de ausencia de publicación. Por eso no se presupone que faltan completamente al azar. Las medianas del preprocesado se ajustan solo en entrenamiento o dentro del fold. Una renta no publicada no es cero y no se inventa como etiqueta.

Dónde comprobarlo: src/ml_data.py · entrega_ml/resultados/diccionario.csv · entrega_ml/resultados/patron_nulos_por_anio.csv · entrega_ml/data/procedencia.json.

14 · Particiones temporales y prevención de fuga

Se usa el pasado para entrenar y un periodo posterior para comprobar. Un reparto aleatorio mezclaría observaciones cercanas de una misma serie y podría dar una imagen demasiado optimista de la previsión.

Partición	Periodo	Filas	Función
Train	2016–2023	601	Ajustar preprocesado y estimadores; buscar hiperparámetros dentro de train.
Validación	2024	75	Comparar familias, elegir modelo y producir SHAP.
Test final	2025 T1–2026 T1	92	Medir una vez el modelo ya seleccionado.

Dentro de train se usa TimeSeriesSplit sobre los **trimestres únicos**, con tres particiones de ventana creciente. Después se trasladan esos índices a las filas municipales. Así un mismo trimestre no queda parcialmente en entrenamiento y parcialmente en validación del fold.

Qué evita fuga de información

El escalado, la imputación y la codificación se ajustan dentro de cada pipeline de entrenamiento. No se utiliza la renta actual, un precio/m² derivado de ella ni una categoría de accesibilidad construida con el objetivo como entrada. Los contratos contemporáneos tampoco son entrada; se utiliza su retardo.

Los retardos de validación y test usan observaciones anteriores disponibles para una predicción a un paso. Eso puede incluir el trimestre anterior del propio periodo de evaluación. No equivale a pronosticar cinco trimestres seguidos sin recibir nuevos datos observados.

Límite del backtest

EMAL está revisado a la fecha de captura. El experimento es retrospectivo sobre esa versión publicada; no demuestra que cada dato estuviera disponible con el mismo valor en la fecha histórica de una predicción real. También hay dependencia temporal y territorial entre observaciones: 768 filas no equivalen a 768 casos independientes de personas.

El test ya está cerrado

Se conservó una evaluación final. Desde entonces se leen el informe y la figura existentes. No se cambia el ganador por sus resultados. Usar historia ya observada para el siguiente caso del formulario, una vez cerrado el experimento, no es volver a evaluar el holdout ni reentrenar el modelo.

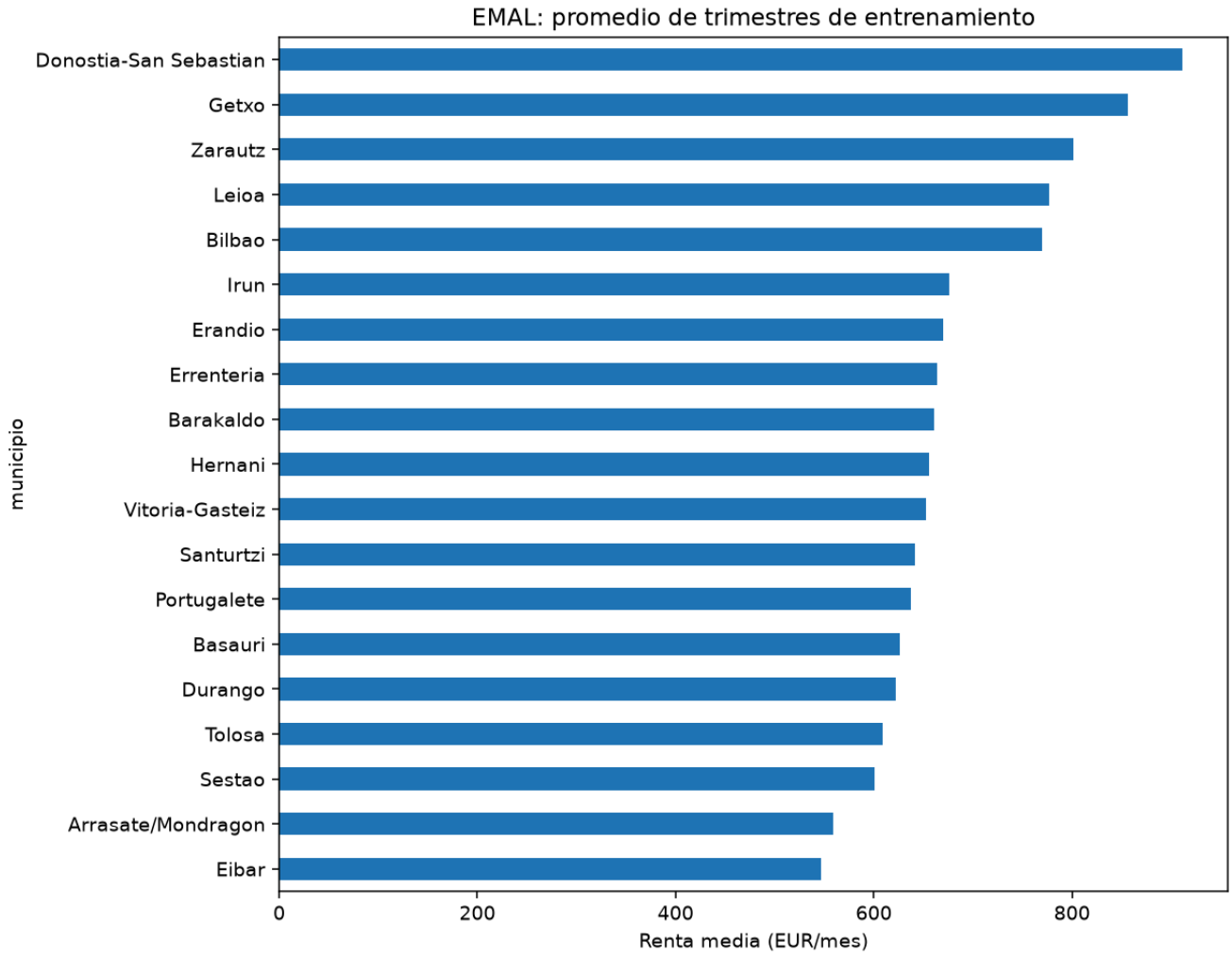
Dónde comprobarlo: src/ml_data.py · src/ml_workflow.py · docs/EVIDENCIA_ML_VALIDACION.md · docs/TEST_FINAL_EMAL.json.

15 · Las cinco gráficas EDA, explicadas una a una

Las cinco figuras siguientes ya estaban calculadas sobre train. Este manual las incorpora y explica; no las recalcula con validación ni test. Para cada una debéis identificar ejes, unidad, patrón, decisión y límite.

Dónde comprobarlo: src/ml_workflow.py · entrega_ml/resultados/conclusiones_eda.json.

15.1 · Barras: diferencias entre municipios



Barras: diferencias entre municipios · figura original conservada de entrenamiento.

Cómo leerla

El eje horizontal expresa euros mensuales y el vertical municipios. Cada barra resume la media de las rentas municipales trimestrales de entrenamiento. Donostia presenta la mayor media, 911,58 €.

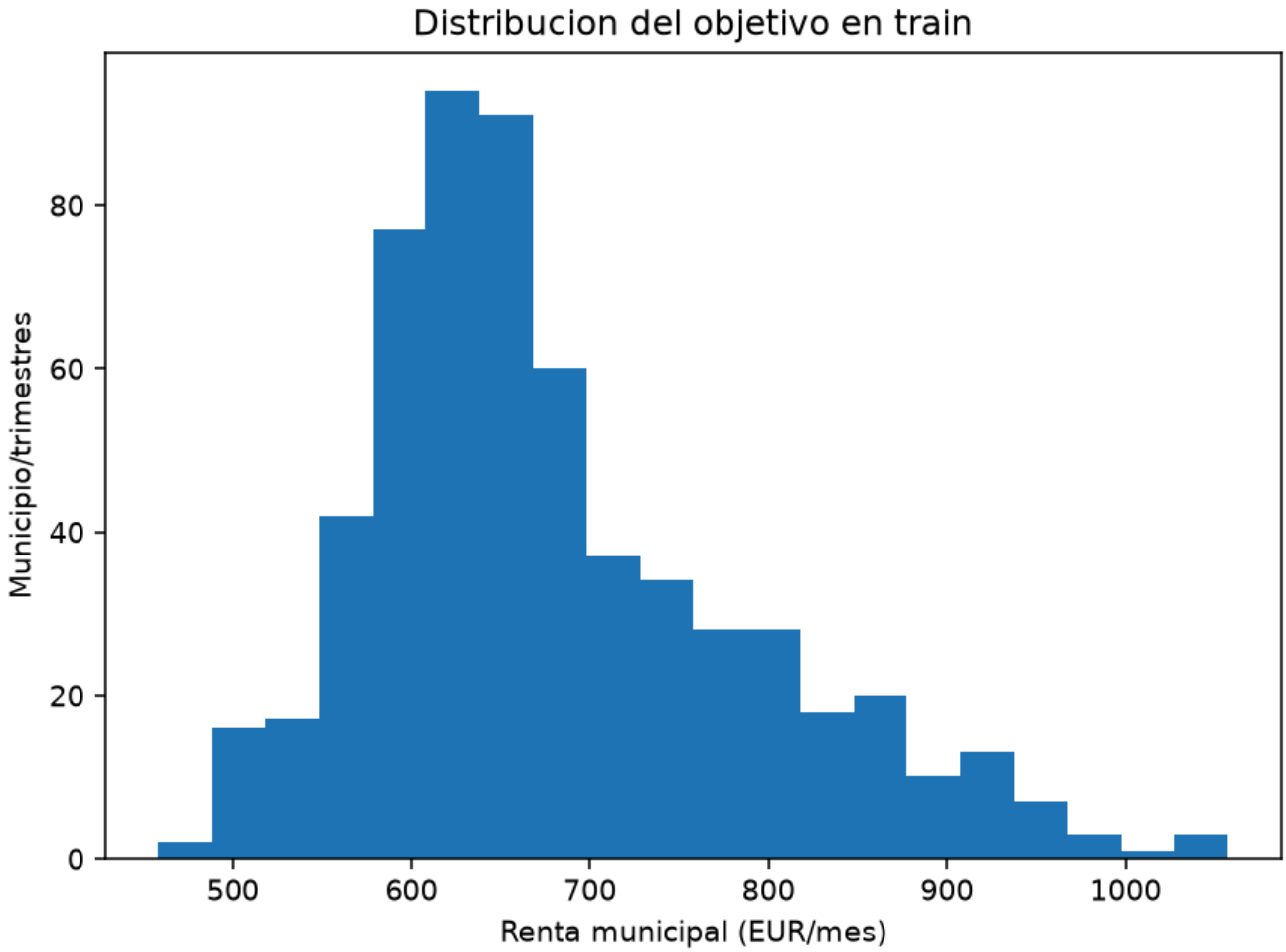
Qué conclusión tomar

Justifica incluir información territorial y comparar municipios. No es una media ponderada por contratos ni un precio de oferta actual. No se interpreta como efecto causal de vivir allí.

Comentario conservado: Donostia-San Sebastian presenta la mayor media de train (911.58 EUR). Es una media de agregados trimestrales, no ponderada por contratos.

Dónde comprobarlo: entrega_ml/resultados/01_barras.png.

15.2 · Histograma: distribución del objetivo



Histograma: distribución del objetivo · figura original conservada de entrenamiento.

Cómo leerla

El eje horizontal agrupa rentas en intervalos; el vertical cuenta observaciones municipio/trimestre. La mediana de train es 656,00 €. No cuenta personas ni viviendas individuales.

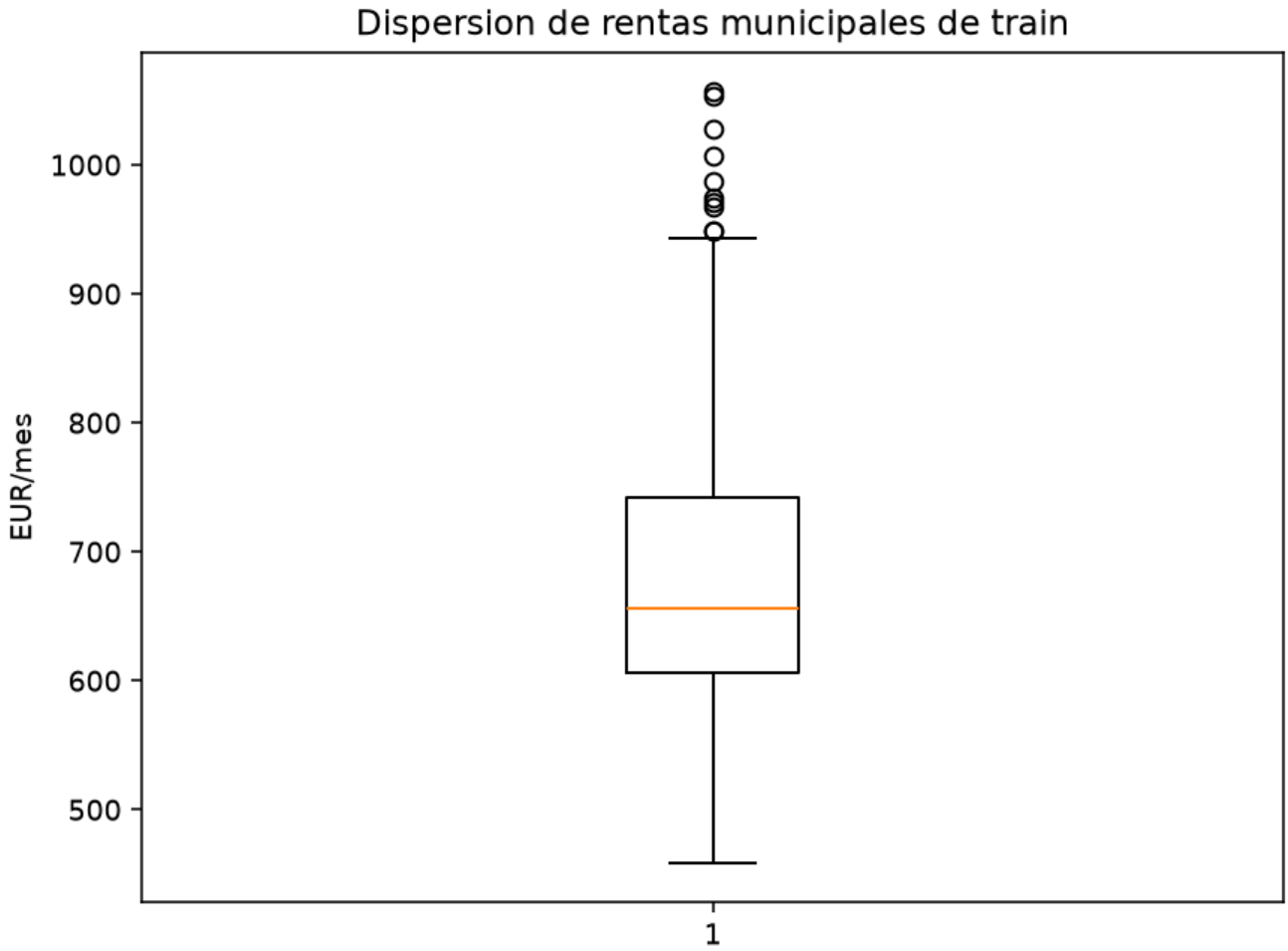
Qué conclusión tomar

Permite entender concentración y cola de valores y contextualizar errores en euros. Los valores altos pueden ser reales; no se eliminan por la forma del histograma.

Comentario conservado: La mediana de train es 656.00 EUR. El objetivo es una renta municipal observada, no la viabilidad de una persona.

Dónde comprobarlo: entrega_ml/resultados/02_histograma.png.

15.3 · Boxplot: dispersión y valores extremos



Boxplot: dispersión y valores extremos · figura original conservada de entrenamiento.

Cómo leerla

La línea central representa mediana; la caja va de Q1 a Q3. El rango intercuartílico es 136,13 €. Los puntos extremos, si aparecen fuera de bigotes, son candidatos a revisión, no errores confirmados.

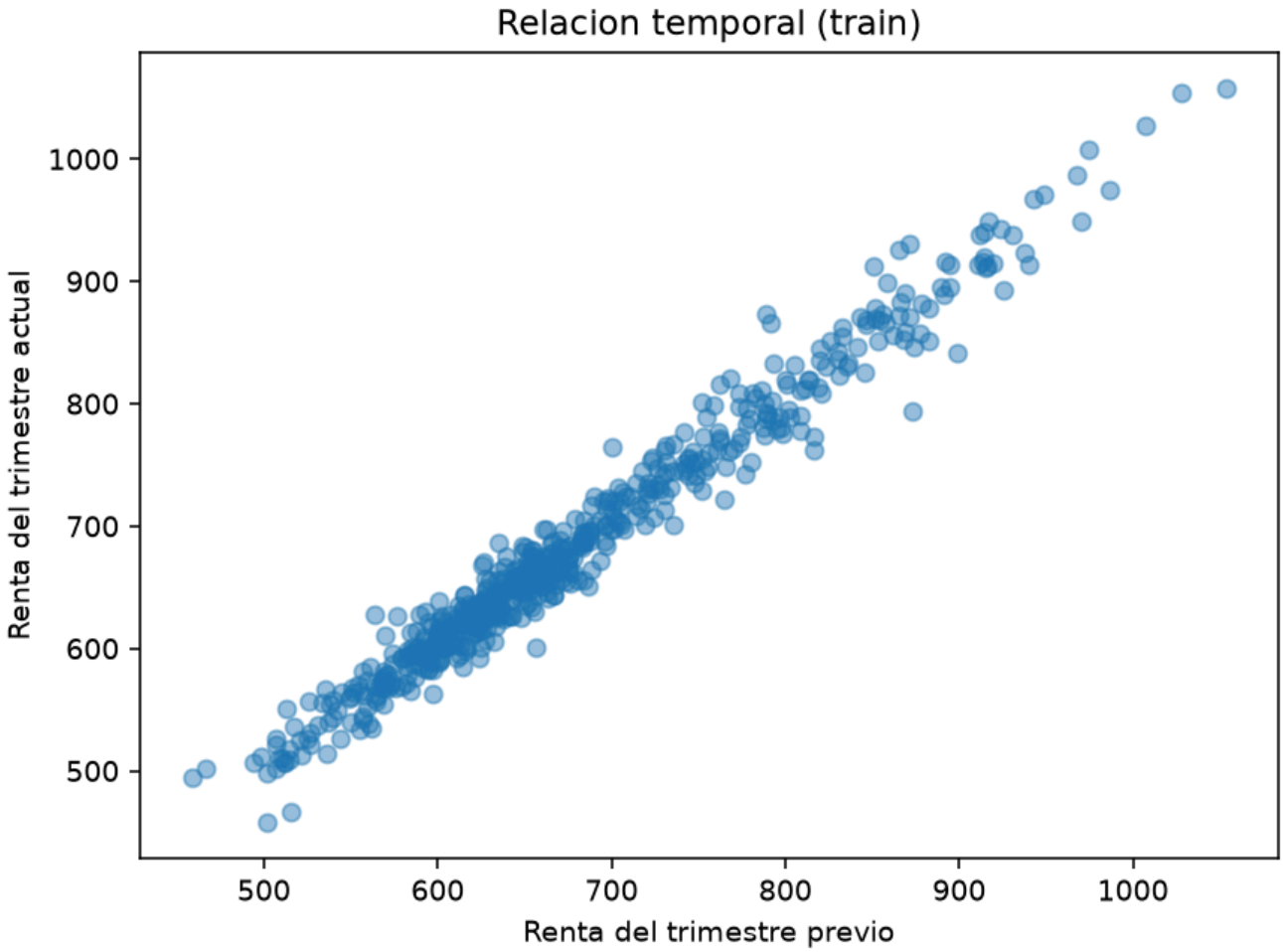
Qué conclusión tomar

Se conserva heterogeneidad territorial real. No se recortan rentas altas automáticamente. El boxplot resume el conjunto de train y no acredita distribución de todos los contratos.

Comentario conservado: IQR = 136.13 EUR. No se eliminan precios altos solo por ser extremos: pueden reflejar diferencias territoriales reales.

Dónde comprobarlo: entrega_ml/resultados/03_boxplot.png.

15.4 · Dispersión: persistencia temporal



Dispersión: persistencia temporal · figura original conservada de entrenamiento.

Cómo leerla

Cada punto relaciona la renta previa en el eje X y la actual en Y. La correlación en train es aproximadamente 0,985, con una relación muy estrecha.

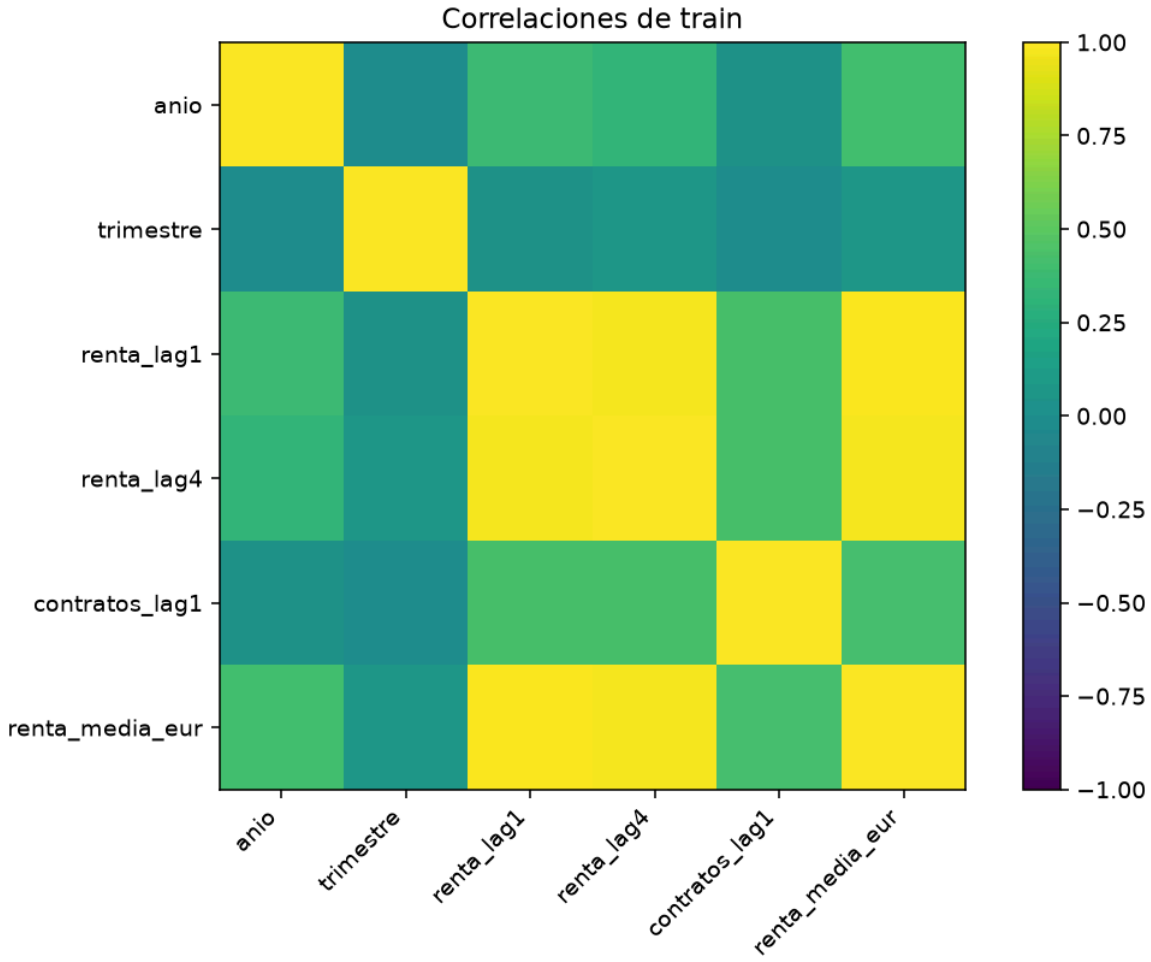
Qué conclusión tomar

Obliga a comparar contra persistencia: predecir la misma renta que en t−1. Una red puede parecer muy precisa por esa continuidad; la mejora debe medirse contra ese baseline. Correlación no implica causalidad.

Comentario conservado: Correlacion lag1/objetivo = 0.985. La persistencia debe ser un baseline; correlacion no demuestra causalidad.

Dónde comprobarlo: entrega_ml/resultados/04_dispersion.png.

15.5 · Heatmap: relaciones entre variables numéricas



Heatmap: relaciones entre variables numéricas · figura original conservada de entrenamiento.

Cómo leerla

Cada celda muestra correlación lineal entre dos columnas. El color representa signo e intensidad entre −1 y 1. La diagonal vale 1 por definición. Incluye el objetivo para explorar relaciones.

Qué conclusión tomar

Los retardos recogen persistencia. Incluir el objetivo en una gráfica exploratoria no significa introducirlo como feature. La correlación no identifica relaciones no lineales ni prueba causa; la matriz tampoco mide la contribución individual del modelo.

Comentario conservado: Los retardos recogen persistencia. Ni la renta actual, ni su precio/m2, ni una categoria derivada del objetivo se incluyen entre las entradas.

Dónde comprobarlo: entrega_ml/resultados/05_correlacion.png.

16 · Pipeline y familias de modelos: por qué cada pieza

El pipeline encadena transformaciones y estimador. Así el mismo objeto que se ajusta durante entrenamiento conserva los pasos necesarios para un caso nuevo. Evita repetir a mano un preprocesado distinto en cada notebook o aplicación.

Preprocesado común

ColumnTransformer separa numéricas y categóricas. Las numéricas se imputan con mediana y escalan con StandardScaler; las categorías se imputan con la categoría más frecuente y codifican con OneHotEncoder. handle_unknown=ignore evita que una categoría desconocida provoque por sí sola un error de codificación; no convierte esa categoría en un caso validado por la cobertura del negocio. El escalador resta media y divide por desviación aprendidas de train: no convierte todos los datos a 0–1.

El código municipal es un identificador, no una cantidad: su distancia numérica no mide cercanía geográfica. OneHotEncoder evita tratar 48020 como «mayor vivienda» que 01059. La provincia también es categórica. Los árboles necesitan menos el escalado, pero comparten el preprocesado para mantener un recorrido común y comparable.

TransformedTargetRegressor escala la variable objetivo al ajustar y devuelve las predicciones a euros. Por tanto, MAE y RMSE se expresan en euros de renta municipal, no en unidades estandarizadas. El pipeline completo se conserva en best_model.joblib.

Familia	Idea	Motivo de comparación y límite
Regresión lineal	Combina entradas mediante coeficientes	Referencia sencilla y rápida; sus relaciones son limitadas por la forma lineal en variables transformadas.
KNN	Promedia vecinos según distancia	Comprueba una solución basada en similitud; sensible a escalado, categorías y cantidad de vecinos.
Random Forest	Promedia árboles entrenados con variación aleatoria	Captura interacciones no lineales; puede sobreajustar y no extrapola tendencias como una ley temporal.
XGBoost	Añade árboles que corrigen errores previos	Captura relaciones complejas; profundidad y capacidad deben controlarse.
SVR	Ajusta una función con margen y tolerancia epsilon	Alternativa no lineal sensible a escala y C; no es el SVC clasificador de otros ejercicios.
MLP	Capas densas con activaciones no lineales	Permite estudiar capacidad, activación y regularización; exige explicar ajuste y generalización.

GridSearchCV prueba combinaciones dentro de train con CV temporal. Las búsquedas conservadas son pequeñas: KNN k=3/7; Random Forest hojas mínimas 1/3; XGBoost profundidad 2/3; SVR C=1/10. No son una búsqueda exhaustiva de todos los hiperparámetros posibles. El scoring usa MAE negativo porque sklearn maximiza puntuaciones; menor MAE real sigue siendo mejor.

Dónde comprobarlo: src/pipeline.py · src/ml_workflow.py · src/ml_models.py · src/ml_data.py.

17 · MLP: base y tres experimentos razonados

Una MLP es una red de perceptrones multicapa: transforma las entradas en capas ocultas y aprende pesos para aproximar el objetivo. En regresión la salida es continua. El proyecto usa MLPRegressor de sklearn; no se debe decir que se implementó con Keras.

Experimento	Configuración definitoria	Pregunta que comprueba
MLP_base	Una capa oculta de 32 neuronas	¿Qué resultado obtiene una red pequeña como punto de partida?
MLP_2_capas	Capas de 64 y 32 neuronas	¿Aumentar capacidad mejora validación o solo el ajuste de train?
MLP_tanh	Capa de 32 con activación tanh	¿Cambiar la función no lineal ayuda en este conjunto?
MLP_regularizada	Capa de 32 con alpha=0,01	¿Penalizar más los pesos reduce sobreajuste y mejora validación?

Las configuraciones fijan max_iter=1000 y random_state=42. Los valores predeterminados aplicables de sklearn son activación ReLU y optimizador Adam; el experimento tanh cambia la activación. Una neurona combina entradas y pesos, añade un sesgo y aplica activación; el entrenamiento ajusta pesos para reducir la pérdida, propagando gradientes hacia las capas anteriores. Alpha controla regularización L2: no es una probabilidad ni la tasa de aprendizaje. El máximo de iteraciones no garantiza convergencia; los avisos conservados se reconocen y no se ocultan.

No se afirma que la red tenga convoluciones, aprendizaje profundo sobre imágenes ni búsqueda de arquitectura automática. Tampoco se presentan curvas de entrenamiento de Keras como si pertenecieran a este estimador.

Qué ocurrió

MLP_2_capas obtuvo el menor MAE de validación, 24,20 €. La regularizada obtuvo 25,42 €; la base, 25,78 €; tanh, 35,16 €. Más complejidad no garantiza mejorar, y cambiar una activación puede empeorar. Se conserva la comparación completa, sin eliminar experimentos desfavorables.

Diferencia con clase

Los notebooks de redes del ZIP usan Sequential, Dense y Adam en Keras. Comparten el concepto de capas densas y experimentación, pero no las llamadas ni todos los parámetros. La rúbrica exige MLP base y tres experimentos; no obliga a una biblioteca. En una pregunta individual hay que enseñar la factoría de modelos y reconocer esa diferencia con claridad.

Dónde comprobarlo: src/ml_models.py · entrega_ml/resultados/comparativa_mlflow.csv.

18 · Métricas, comparación y elección del ganador

Qué significa cada métrica

MAE es la media de los errores absolutos: si predigo 900 y el observado es 930, ese caso aporta 30 € al error absoluto. RMSE eleva errores al cuadrado antes de promediar y volver a la raíz, por lo que penaliza más los errores grandes. R^2 compara con predecir la media del conjunto evaluado; puede ser negativo y **no es porcentaje de aciertos**.

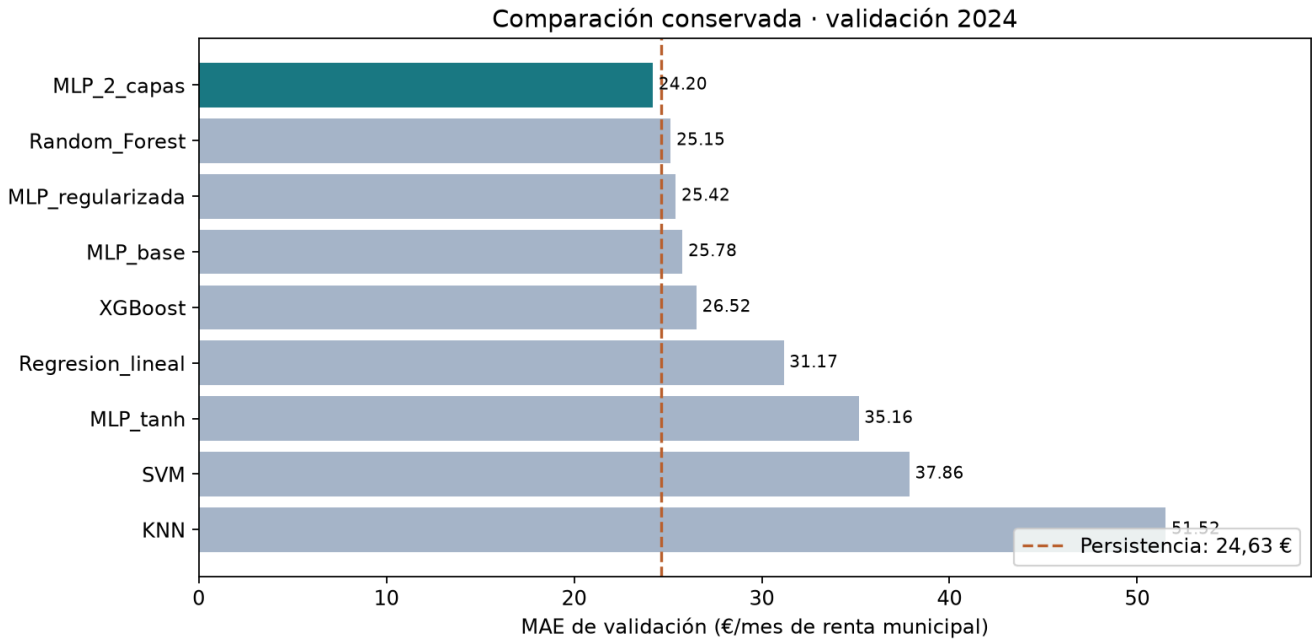
```
MAE = promedio de |observado - predicho|
RMSE = raíz del promedio de (observado - predicho)²
R² = 1 - suma de errores² / suma de desviaciones respecto a la media²
```

Se prioriza MAE porque expresa en euros el error de la referencia municipal utilizada para orientar. RMSE permite vigilar errores grandes y R^2 aporta contexto adicional. Ninguna métrica representa por sí sola el riesgo de impago de una persona.

Tabla conservada de MLflow

Los tiempos son históricos e incluyen el ajuste o búsqueda de cada procedimiento. Dependen de la máquina y del tamaño de la búsqueda; no representan la latencia de la aplicación.

Modelo	MAE train	RMSE train	R^2 train	MAE val	RMSE val	R^2 val	Segundos
MLP_2_capas	15.8800	22.3526	0.9558	24.1970	31.8597	0.9370	0.0799
Random_Forest	5.9217	9.0960	0.9927	25.1507	34.7620	0.9250	1.8462
MLP_regularizada	14.7771	21.2229	0.9601	25.4173	34.0273	0.9282	0.1250
MLP_base	14.2235	19.8385	0.9652	25.7799	34.3755	0.9267	0.1260
XGBoost	11.6437	16.3124	0.9765	26.5234	37.0516	0.9148	0.2494
Regresion_lineal	12.7637	16.6719	0.9754	31.1665	40.5837	0.8978	0.2258
MLP_tanh	16.9724	23.4404	0.9514	35.1592	50.7876	0.8400	0.1391
SVM	10.6001	16.5865	0.9757	37.8630	55.5557	0.8085	0.1856
KNN	0.0000	0.0000	1.0000	51.5196	60.8912	0.7700	0.0917



Resumen gráfico de los MAE de validación existentes; línea de persistencia conservada.

Por qué se conserva el MLP de dos capas

Tiene el menor MAE de validación, 24,1970 €, frente a 24,6267 € de persistencia. La diferencia es 0,4296 €, aproximadamente 1,74 % del MAE del baseline; no se ha demostrado significación estadística ni superioridad generalizable. El procedimiento contempla preferencia por regresión lineal si queda dentro del 1 % del mejor MAE; no ocurre aquí.

KNN tiene MAE train cero y MAE val 51,52 €. Los vecinos ponderados por distancia pueden memorizar el entrenamiento; no demuestra que generalice. Random Forest también muestra una diferencia notable entre train y validación. La comparación evita elegir únicamente por ajuste a los datos vistos.

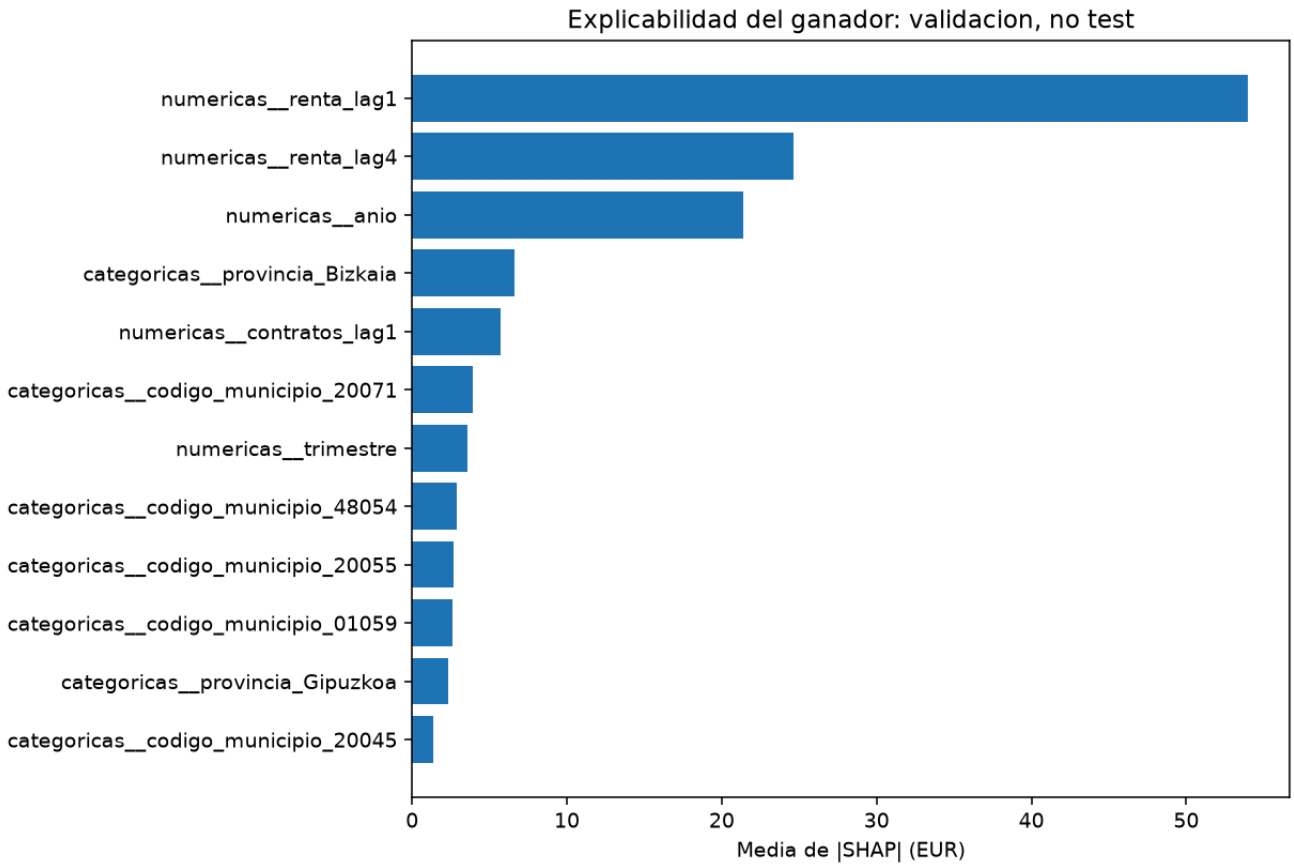
La decisión publicada no se modifica después de consultar el test. El modelo es una ayuda experimental para una referencia agregada; no se presenta como una herramienta de tasación certificada.

Dónde comprobarlo: entrega_ml/resultados/comparativa_mlflow.csv · entrega_ml/models/selection.json · src/ml_workflow.py.

19 · SHAP, SMOTE y test final: interpretar sin exagerar

SHAP en validación

SHAP aproxima contribuciones de variables a las predicciones del modelo. Aquí se usa el algoritmo permutation sobre 30 filas de validación y un fondo de 30 filas de entrenamiento, en euros. No se utiliza test. La media del valor absoluto expresa magnitud media de contribución en esa muestra; no el signo del efecto ni un porcentaje causal.



SHAP original conservado: validación, fondo de train, interpretación no causal.

Las mayores magnitudes medias son renta_lag1, aproximadamente 53,97 €, renta_lag4, 24,66 €, y año, 21,40 €. Eso es coherente con persistencia y tendencia. No significa que subir la renta anterior cause necesariamente una subida de 53,97 €. Las categorías codificadas tienen columnas propias; una categoría individual no equivale a toda la importancia de un municipio.

SMOTE es un experimento auxiliar

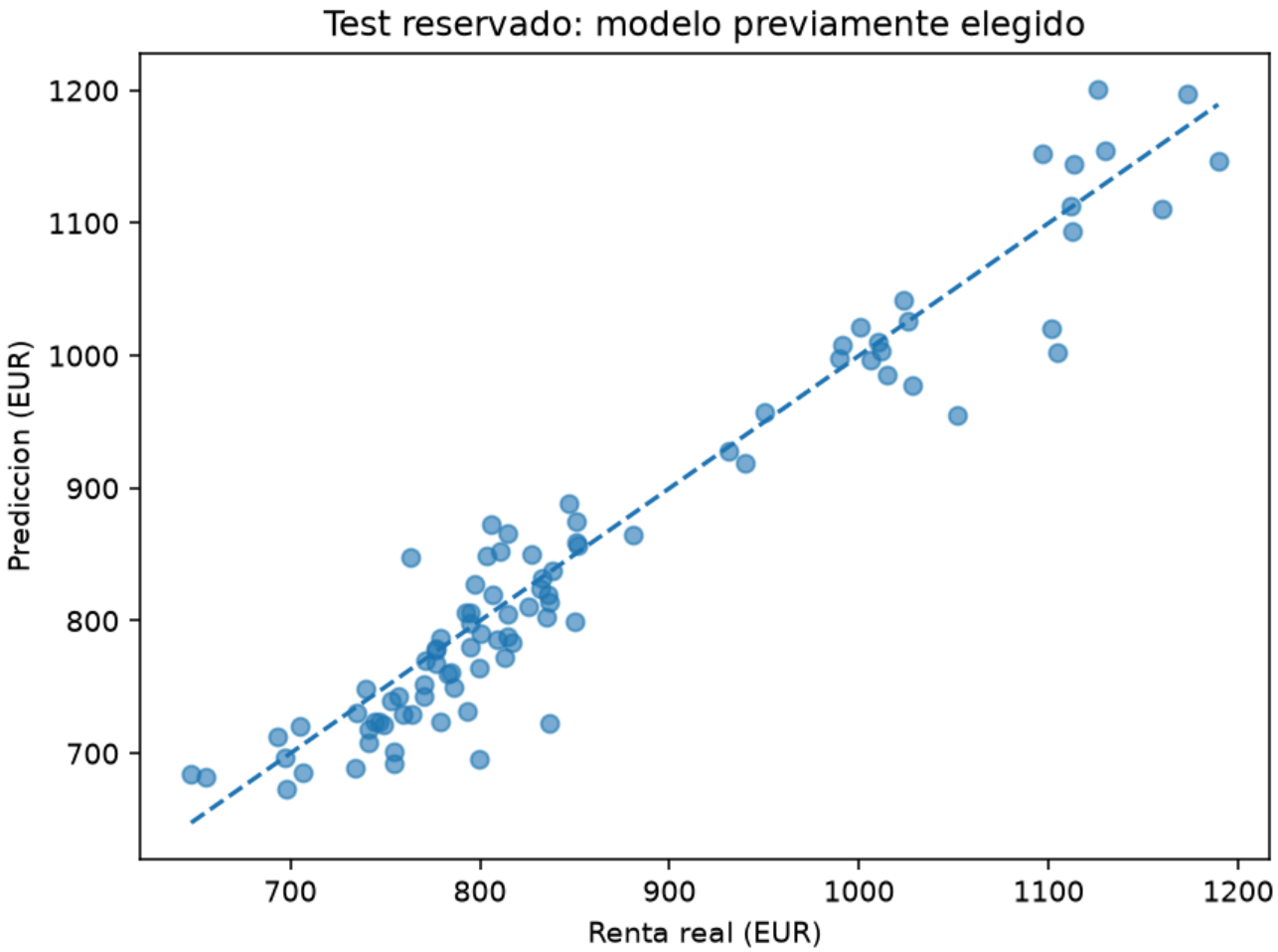
La renta es continua; no corresponde balancearla con SMOTE. Para demostrar el procedimiento se creó una tarea separada: clasificar si la renta observada sube frente al trimestre anterior. Solo usa variables numéricas, con imputación y escalado, y el balanceo se aplica en entrenamiento. Las categorías se excluyen para no interpolar códigos one-hot como si fueran cantidades continuas. Tras excluir lag1 ausente quedan 576 filas de train: 357 subidas y 219 no subidas.

Validación auxiliar	Sin SMOTE	Con SMOTE
Precisión	0,7123	0,7313
Recall	0,9811	0,9245
F1	0,8254	0,8167
TN / FP / FN / TP	1 / 21 / 1 / 52	4 / 18 / 4 / 49

La clase positiva es subida de renta. SMOTE mejora algunos aspectos, pero reduce F1. No se afirma que mejore globalmente. Las matrices son de esta clasificación auxiliar, no del modelo de regresión elegido ni de personas viables/no viables.

Test final conservado

Sobre 92 observaciones, MAE=28,6656 €, RMSE=37,9285 € y R^2=0,9189. El error absoluto medio no es un intervalo garantizado de ±28,67 € para cada caso. Puede haber errores mayores y no se publica un intervalo predictivo calibrado.



Valores reales frente a predichos de la única evaluación final conservada.

En el gráfico, cercanía a la diagonal indica predicciones próximas al valor real. La dispersión alrededor de ella representa error. No basta mirar un R^2 alto para decir que el sistema acierta el 91,89 % de las veces.

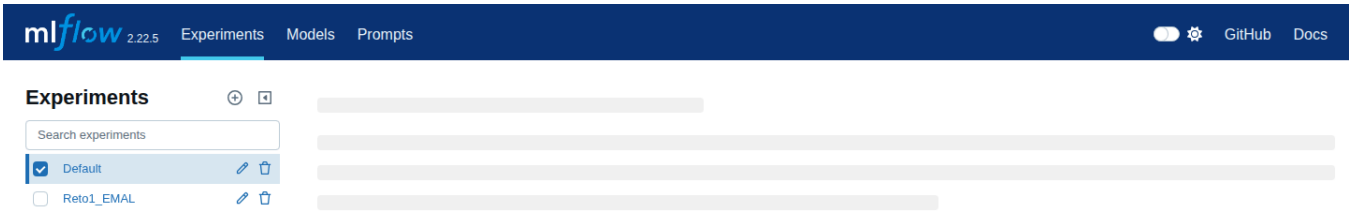
selection.json conserva el estado histórico de selección anterior a la evaluación; el informe final y su contador de una evaluación acreditan el estado actual. La aplicación comprueba huellas y reutiliza el informe. Este manual no carga joblib ni ejecuta predicciones del test.

Dónde comprobarlo: entrega_ml/resultados/shap_metadatos.json · entrega_ml/resultados/shap_validacion.csv · entrega_ml/resultados/comparacion_smote.json · docs/TEST_FINAL_EMAL.json · src/ml_workflow.py.

20 · MLflow, SQLite y artefactos del modelo

MLflow registra experimentos: nombre de ejecución, hiperparámetros, métricas, tiempo y artefactos. Un run no es un municipio ni una fila del dataset. Se conservan nueve ejecuciones de regresión y dos de la comparación auxiliar SMOTE: once en total.

La base de metadatos está en entrega_ml/mlruns/mlflow.db y usa SQLite. Los archivos de modelos y otros artefactos están en su directorio de artefactos. SQLite es una base relacional local en archivo; aquí no sustituye a MongoDB para los documentos de vivienda.



Captura histórica real de MLflow; corresponde a los experimentos conservados, no a un entrenamiento nuevo.

Cómo abrirlo sin entrenar

Desde la raíz, con el entorno activo:

```
python -m mlflow ui --backend-store-uri sqlite:///entrega_ml/mlruns/mlflow.db --host 127.0.0.1 --port 5000
```

Abrir http://127.0.0.1:5000 y seleccionar el experimento. Mostrar nombres, métricas y parámetros; filtrar regresión al comparar las nueve filas del modelo principal. La captura y el CSV permiten explicar el resultado aunque no esté abierto el servidor.

Al trasladar MLflow, algunas rutas de artefactos conservadas pueden ser absolutas del entorno original. Debe comprobarse la navegación en el equipo de defensa. Una ruta rota de un artefacto no debe resolverse entrenando otra vez. El pipeline de inferencia independiente está en entrega_ml/models/best_model.joblib y la tabla en entrega_ml/resultados/comparativa_mlflow.csv.

Persistencia y confianza

joblib serializa el objeto Python ya ajustado, incluidas transformaciones. Se carga únicamente el artefacto conocido y verificado. El archivo no es una descripción JSON de una red ni un archivo de pesos Keras. Dependencias y versiones importan para deserializarlo; usar el entorno fijado y comprobar la huella antes de abrir la app.

Dónde comprobarlo: src/mlflow_tracker.py · entrega_ml/models/selection.json · requirements.txt · app/streamlit_app.py.

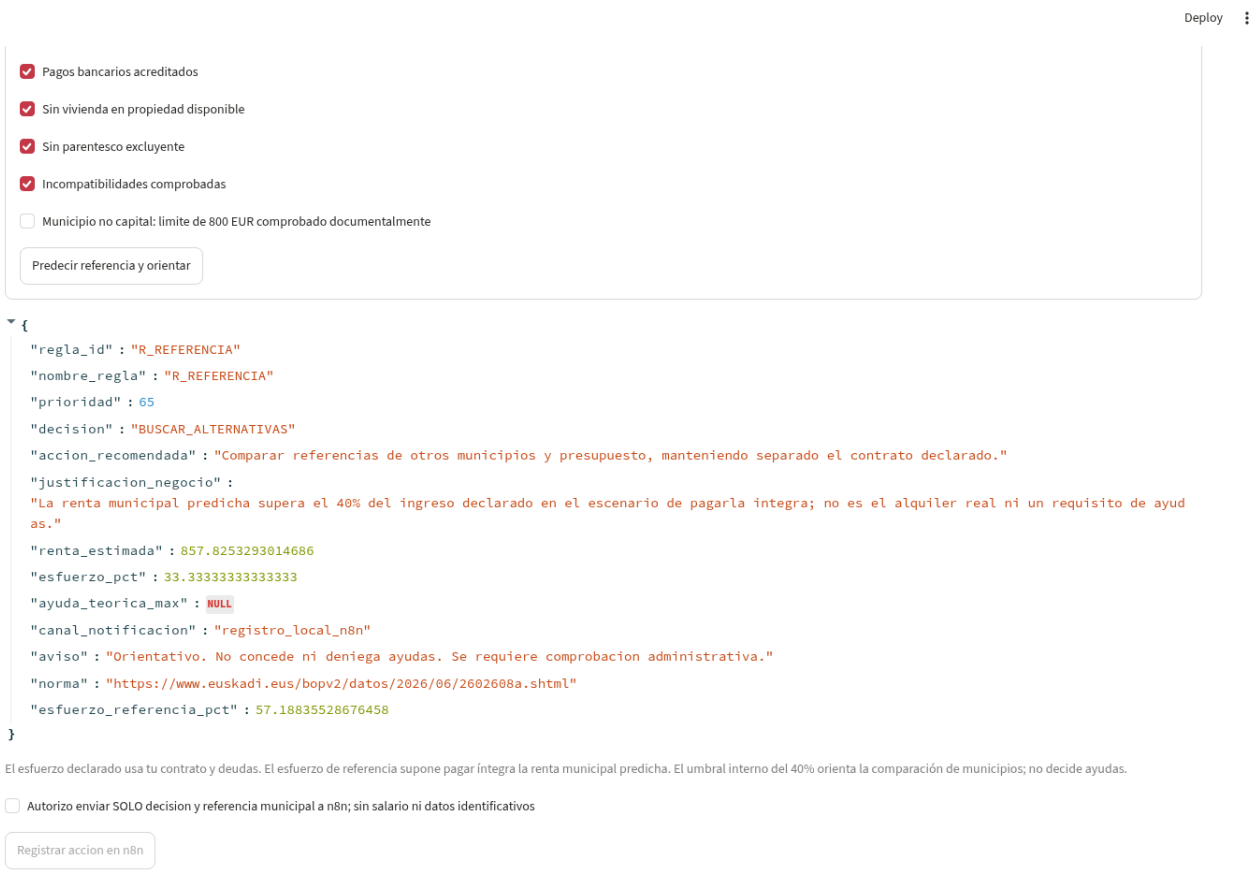
21 · Streamlit: qué hace cada pantalla

Streamlit convierte Python en una interfaz interactiva. La aplicación del modelo tiene tres pestañas. Carga el pipeline conservado, verifica su huella y usa variables y resultados guardados. No necesita ejecutar el entrenamiento al iniciar.

Pantalla	Qué mostrar	Qué explicar
Modelos y datos	Tabla, selección, persistencia, SHAP y EDA	La elección se hizo con validación; gráficos y tiempos son conservados.
Test reservado	Mensaje de informe conservado, métricas y gráfico	El nombre de pestaña es histórico; el test ya fue evaluado una vez y ahora se lee.
Nuevo caso y accion	Municipio, referencia predicha, perfil, regla y decisión	La predicción es municipal; los ingresos y contrato declarados intervienen después en reglas.

El formulario obtiene retardos de la historia conservada y predice el siguiente periodo. El usuario no introduce libremente la renta municipal objetivo para hacer parecer buena la predicción. Si faltan retardos, se aplica el preprocesado del pipeline; un municipio fuera de cobertura no se considera un caso válido sin más.

El estado de sesión conserva el identificador del evento y permite evitar duplicados al repetir la misma acción. La interfaz exige consentimiento antes del envío. La respuesta debe confirmar persistencia; un HTTP exitoso sin ese contrato no basta.



Caso inventado de demostración: referencia del modelo conservado y decisión R_REFERENCIA.

El visor territorial es otra app, en app/territorio.py. Lee resultados de MongoDB previamente exportados y verifica sus huellas. Poder abrir ese visor sin MongoDB activo demuestra que la vista archivada es accesible, no que las consultas se estén ejecutando en vivo en ese instante. El mapa utiliza cartografía externa y puede necesitar red aunque los resultados sean locales.

Dónde comprobarlo: app/streamlit_app.py · app/territorio.py · tests/test_project_launcher.py.

22 · Reglas MIA: prioridades, motivos y parámetros

El motor construye un contexto y selecciona **la primera regla aplicable en prioridad descendente**. No es un modelo aprendido ni un sistema que concede derechos. Varias reglas pueden compartir decisión; por eso diez reglas producen ocho decisiones y ramas, no diez ramas.

Regla	Prioridad	Condición principal	Decisión
R_SEGURIDAD	100	Datos inválidos o cobertura no válida	REVISION_DATOS
R_NORMA	95	Fecha fuera de 2026	REVISION_NORMATIVA
R_ESFUERZO_CRITICO	90	Esfuerzo declarado >70 %	ORIENTACION_PRIORITARIA
R_REQUISITOS	80	Requisito básico declarado que requiere revisión	REVISION_REQUISITOS
R_DOCUMENTACION	75	Comprobaciones o excepción pendientes	REVISION_NORMATIVA
R_ALTERNATIVAS	70	Esfuerzo declarado >40 %	BUSCAR_ALTERNATIVAS
R_REFERENCIA	65	Referencia municipal predicha / ingreso >40 %	BUSCAR_ALTERNATIVAS
R_ENTRADA	60	Ahorro menor que coste de entrada declarado	PLANIFICAR_ENTRADA
R_GAZTELAGUN	50	Sin reglas anteriores y esfuerzo declarado >30 %	CONSULTAR_GAZTELAGUN
R_GENERAL	0	Caso restante	INFORMACION_GENERAL

Dos esfuerzos distintos

```
esfuerzo_declarado = (pago_real_alquiler + deudas) / ingreso_neto × 100
esfuerzo_referencia = renta_municipal_predicha / ingreso_neto × 100
```

El primero usa el contrato y deudas declarados. El segundo supone pagar íntegra la referencia municipal; no reemplaza el contrato. El 40 % y 70 % son umbrales internos de orientación. El umbral descriptivo territorial del 30 % y las condiciones legales de ayudas tienen funciones diferentes; no deben confundirse.

Parámetros normativos conservados

El código referencia la Orden de 5/6/2026, publicada el 15/6/2026. Contempla edad de 18 a 35, ingresos computables anuales mayores que 3.000 y máximos parametrizados de 30.000, 36.764 o 39.184 según unidad. La renta de vivienda revisada usa 900 € para las tres capitales y 800 € solo cuando el grupo municipal está verificado; contratos compartidos o excepciones pasan a revisión.

Los ingresos anuales computables no se deducen automáticamente de salario neto mensual ×12. Se solicitan por separado. Se comprueban declaraciones sobre residencia, titularidad, pagos, propiedad, parentesco e incompatibilidades. Aun con todas completas, se recomienda consultar; no se acredita jurídicamente su verdad.

Solo para CONSULTAR_GAZTELAGUN se muestra una cuantía teórica de min(300 €, 60 % del pago declarado), sujeta a comprobación oficial. Fuera del año parametrizado la regla normativa tiene prioridad. Esto documenta lo implementado; no sustituye la revisión de la convocatoria vigente.

Dónde comprobarlo: src/reglas.py · tests/test_business_rules.py.

23 · De la predicción a n8n: prueba y acción real

La prueba contrafactual acredita modelo → decisión sobre un mismo perfil inventado: ingreso neto 1.500 €, alquiler 500 €, deuda cero, ingreso anual computable 18.000 € y comprobaciones declaradas completas.

Valor de prueba	Esfuerzo declarado	Esfuerzo de referencia	Decisión
Referencia 600 €	33,33 %	40,00 %	CONSULTAR_GAZTELAGUN
Referencia 1.600 €	33,33 %	106,67 %	BUSCAR_ALTERNATIVAS por R_REFERENCIA

600 y 1.600 son valores introducidos en una prueba del motor, no resultados atribuidos al modelo. La comparación demuestra que cambiar la referencia puede cambiar la decisión sin cambiar el contrato. En la prueba completa de la app, el modelo conservado produjo 857,8253 €, referencia de esfuerzo 57,1884 %, y se guardó una ficha de alternativas. Ese perfil también es inventado.

Flujo de n8n

Webhook POST → switch de decisión → petición HTTP a receptor local → confirmación de persistencia. Las ocho acciones son:

Decisión	Acción persistida
REVISION_DATOS	revision_datos
REVISION_NORMATIVA	revision_normativa
ORIENTACION_PRIORITARIA	cita_prioritaria: solicitud interna, pendiente de asignación humana
REVISION_REQUISITOS	checklist_requisitos
BUSCAR_ALTERNATIVAS	ficha_alternativas
PLANIFICAR_ENTRADA	plan_entrada
CONSULTAR_GAZTELAGUN	checklist_gaztelagun
INFORMACION_GENERAL	ficha_informativa

El nombre «cita_prioritaria» no significa una reserva real en Google Calendar. Cada acción crea una ficha local distinta según decisión, con regla y referencia. No se envían correos, SMS ni solicitudes administrativas.

Payload mínimo e idempotencia

Se envían event_id (UUID), decisión, regla_id, renta_estimada, origen y versión. No se envían nombre, salario, dirección ni el perfil completo. El receptor verifica coherencia entre ruta y decisión. Un UUID repetido con el mismo contenido devuelve la acción existente; si el contenido cambia, se rechaza el conflicto.

El éxito exige event_id coincidente y persistido=true; un fallo produce ERROR_N8N. Compose conserva el estado de n8n en su volumen y las fichas en data/acciones_n8n. Es una demostración local.

Dónde comprobarlo: src/n8n.py · n8n/flujo.json · n8n/compose.yml · scripts/action_server.py · docs/evidencias_cierre_rubrica_2026-11-12/app-n8n.json.

24 · Herramientas utilizadas y relación con clase

Inventario de herramientas con función efectiva

Herramienta	Para qué se utiliza
Python 3.12; compatibilidad prevista 3.13	Scripts, servicios clientes, análisis y apps.
uv, uv.lock, pip, venv	Resolver o instalar dependencias y aislar el entorno; no son modelos.
requests; urllib y robotparser	HTTP, URLs y normas de acceso del scraper.
BeautifulSoup	Interpretar HTML de listados y fuentes.
Pydantic	Esquemas y validación de registros.
pandas, NumPy, openpyxl	Tablas, operaciones numéricas y lectura XLSX.
Hadoop / WebHDFS / Java 11	Almacenamiento y recuperación HDFS del recorrido docente.
Apache Kafka; kafka-python-ng	Broker y productor/consumidor Python.
MongoDB / PyMongo	Base documental y consultas/agregaciones.
mongomock y adaptadores simulados	Pruebas aisladas; no acreditan planes de ejecución reales.
scikit-learn; XGBoost	Preprocesado, CV, cinco familias y MLP.
imbalanced-learn	SMOTE auxiliar dentro de entrenamiento.
SHAP	Explicación aproximada del modelo en validación.
matplotlib	Cinco EDA, SHAP y error final; gráficas explicativas del manual leen resultados existentes.
MLflow / SQLite / SQLAlchemy	Registro de experimentos y metadatos; SQLAlchemy tiene límite de versión por compatibilidad.
joblib y serialización asociada	Pipeline de inferencia guardado.
Streamlit	Formulario y visualización de modelo y territorio.
n8n / Docker Compose / receptor HTTP Python	Ramas de acción y fichas locales.
pytest, nbformat, nbclient, ipykernel	Pruebas y notebooks ejecutables.
Git / GitHub / Actions	Versiones, integración y comprobaciones en máquinas separadas.
Chromium / Playwright	Comprobación visual, capturas y exportación PDF del material de estudio.

Comparación con los ZIP

Se inventariaron 49 archivos de EJERCICIOS.zip y 195 de ejercicios2.zip, incluidos 12 y 59 notebooks respectivamente. Las prácticas relevantes contienen requests, BeautifulSoup, pandas, HDFS/Kafka, KNN, ensembles, redes densas, validación, guardado de pipelines, Streamlit y reglas de gimnasio.

Los fundamentos son los mismos. El proyecto añade integración, procedencia, hashes, idempotencia, separación de unidades, evidencias de servicios y test cerrado. Esas piezas aumentan el trabajo de ingeniería; no implican que cada algoritmo sea ajeno al temario.

Diferencia concreta	Explicación que debéis dar
Keras de clase frente a MLPRegressor	Misma familia MLP; APIs distintas. No se importó una red Keras del ZIP.
confluent_kafka frente a kafka-python-ng	Clientes diferentes del servicio Kafka; explicar el usado.
hdfs.InsecureClient frente a REST WebHDFS	Dos formas de acceder a HDFS; aquí se muestran operaciones HTTP.
Reglas encadenadas de algunos ejercicios frente a primera coincidencia	El proyecto calcula contexto y selecciona por prioridad; la rúbrica no obliga a encadenamiento.
Validación temporal	TimeSeriesSplit ya aparece en el PDF de clase; no se vende como novedad no vista.
SHAP y SMOTE auxiliar	Amplían los ejemplos aportados; no garantizan por sí solos la novedad respecto a todo el curso.

Los ZIP no aportan prácticas suficientes de MongoDB o flujos n8n para certificar ese nivel comparativo ni la novedad de la acción. No se puede garantizar la puntuación máxima de esos criterios solo con estos archivos.

Código histórico que no acredita la versión actual

Hay clientes y demostraciones heredados, incluida una ruta Udalmap sin captura integral acreditada, constantes antiguas del cliente OVJ, clasificación de perfiles sintéticos y JSON de resultados históricos. Que un módulo exista no significa que su resultado forme parte de la evidencia vigente. El recorrido defendido es el descrito en este manual y los artefactos conservados; no se presentan esas constantes como observaciones nuevas.

Dónde comprobarlo: requirements.txt · pyproject.toml · uv.lock · docs/CONTRASTE_DOCENTE.md · docs/BDA_ESTADO.md.

25 · Preparar el ordenador sin volver a entrenar

Usad la raíz del repositorio como directorio de trabajo. La app del modelo y el visor pueden abrir resultados conservados sin arrancar todos los servicios Big Data; una demostración de transporte real sí requiere HDFS, Kafka y MongoDB.

Entorno con uv

```
uv sync --locked --extra dev
uv run --no-sync python main.py --audit --json
uv run --no-sync python scripts/sync_sources.py --check
```

El extra dev incluye mongomock para las pruebas. No mezclar paquetes de otro Python. Alternativa con venv: crear un entorno Python 3.12, activarlo e instalar requirements-ci.txt. En Windows se puede invocar directamente .venv/Scripts/python.exe; en Linux, .venv/bin/python. Desde un terminal uv el comando anterior funciona sin activar manualmente.

Abrir las dos apps en terminales diferentes

```
uv run --no-sync python main.py --subject saa_pia_mia --server.port 8501
uv run --no-sync python main.py --subject bda_sbd --visual --server.port 8502
```

Abrir http://localhost:8501 y http://localhost:8502. Si un puerto está ocupado, detener la instancia propia anterior o elegir otro y anotarlo. No interpretar una app que sigue abierta con código anterior como evidencia de la versión recién cambiada; reiniciarla si se ha actualizado el código.

Arrancar n8n local con Docker disponible

```
docker compose -f n8n/compose.yml run --rm n8n import:workflow --input=/workflow/flujo.json
docker compose -f n8n/compose.yml run --rm n8n publish:workflow --id=reto1Vivienda2026
docker compose -f n8n/compose.yml up -d
```

El webhook local predeterminado es http://localhost:5678/webhook/vivienda-decision. No confundir la URL de producción publicada con la URL de prueba temporal del editor n8n. El receptor acciones es alcanzable dentro de la red Compose, no como un hostname público del ordenador.

Configuración de servicios

Variable	Valor local típico	Para qué
MONGO_URI	mongodb://127.0.0.1:27017	Conexión del recorrido real.
MONGO_DB / --db-name	Nombre aislado para ensayo	Base objetivo; varios scripts tienen defaults distintos.
HDFS_NAMENODE_HOST	localhost	Host HTTP del NameNode.
HDFS_NAMENODE_PORT	9870	Puerto WebHDFS de la evidencia.
HDFS_USER	hadoop	Usuario docente configurado.

Variable	Valor local típico	Para qué
KAFKA_BOOTSTRAP_SERVERS	localhost:9092	Broker Kafka alcanzable.
N8N_WEBHOOK_URL	URL local del webhook	Envío de acciones tras consentimiento.
RETO1_ARTIFACT_DIR	entrega_ml, por defecto	Ubicación del modelo y resultados conservados.
RETO1_ANALYSIS_DIR	data/analisis_municipal	Resultados territoriales exportados.
RETO1_RENTALS_DIR	Evidencia archivada, por defecto	Cambiar solo para mostrar otra exportación identificada.

El archivo .env.example no contiene todas estas variables y no todos los scripts leen .env automáticamente. Es preferible exportar variables en el terminal que ejecuta cada comando. En PowerShell se usa, por ejemplo, $env:MONGO_URI='mongodb://127.0.0.1:27017'; en bash, export MONGO_URI='mongodb://127.0.0.1:27017'.

No copiar las direcciones privadas 10.22.87.215 de clase sin comprobar acceso y puertos. No ejecutar el script CI de HDFS contra un almacén existente: está diseñado para un clúster efímero y contiene formato inicial con guardas. En Windows, los servicios Big Data requieren entorno preparado —por ejemplo Linux/WSL o el clúster docente—; las pruebas de apps en Windows no acreditan que Hadoop esté instalado allí.

Lo que no hace falta ejecutar

No ejecutar train_official.py, no borrar test_final.json y no recapturar portales para abrir la entrega. La opción --demo no prueba servicios reales. Generar documentación o mostrar métricas conservadas no requiere entrenar.

Dónde comprobarlo: main.py · .env.example · n8n/compose.yml · infra/start_hdfs_ci.sh · README.md.

26 · Demostración técnica y plan ante fallos

La exposición debe mostrar un recorrido corto y verificable. La preparación completa y las recargas se ensayan antes; no se dedica la mayor parte de los 15 minutos a instalar servicios.

Verificación previa sin efectos sobre el modelo

```
uv run --no-sync python main.py --audit --json
uv run --no-sync python scripts/sync_sources.py --check
uv run --no-sync python -m pytest tests/ -q
```

El conjunto tests/ valida lógica y regresiones; no equivale automáticamente a todas las integraciones con servicios. La evidencia del cierre conserva 168 pruebas y 12 integraciones reales, sin omisiones. Los avisos conocidos y el número de pruebas deben citarse con su fecha y XML, no como si se hubieran vuelto a ejecutar en cada apertura del documento.

Transporte real de API archivada

Con HDFS, Kafka y MongoDB accesibles, en una base de ensayo:

```
uv run --no-sync python scripts/transport_rent_api.py --input-dir data/capturas_api_rentas/2026-11-12 --db-name reto1_ensayo_api --report work/ensayo/api-1.json
uv run --no-sync python scripts/transport_rent_api.py --input-dir data/capturas_api_rentas/2026-11-12 --db-name reto1_ensayo_api --report work/ensayo/api-2.json
```

Comprobar tres recibidos, tres documentos, servicios reales, recuperación idéntica y offsets tras persistencia. La segunda ejecución debe mantener tres documentos. La base de ensayo evita confundir su evidencia con la captura original o con la base de la entrega.

Reproducir anuncios y análisis

Crear una copia de trabajo de la captura en una ruta nueva; no modificar el archivo original sellado:

```
uv run --no-sync python -c "import shutil; shutil.copytree('data/capturas_anuncios/2026-11-12', 'work/ensayo/anuncios')"
uv run --no-sync python scripts/run_pipeline.py --snapshot-dir data/capturas_oficiales --raw-dir work/ensayo/anuncios --output-dir work/ensayo/pipeline --db-name reto1_ensayo_anuncios
uv run --no-sync python scripts/analyze_rental_sample.py --capture-dir data/capturas_anuncios/2026-11-12 --pipeline-dir work/ensayo/pipeline --db-name reto1_ensayo_anuncios --output-dir work/ensayo/analisis_anuncios
```

copytree falla si ya existe la ruta: usar otra carpeta de ensayo, no borrar originales. La captura entra por raw-dir junto a los preparados oficiales. Comprobar 638 recibidos/persistidos y la comparación por portales. Para abrir la exportación nueva en el visor, configurar RETO1_RENTALS_DIR con esa ruta y reiniciar su app.

Exportar análisis municipal

```
uv run --no-sync python scripts/analyze_municipal.py --db-name reto1_ensayo_municipal --output-dir work/ensayo/municipal
uv run --no-sync python scripts/transport_municipal.py --input-dir work/ensayo/municipal --db-name reto1_ensayo_municipal --report work/ensayo/municipal-transporte.json
```

Mantener MONGO_URI coherente entre comandos. Las integraciones pytest requieren sus variables: BDA_LIVE=1, TRANSPORT_LIVE=1 y MONGO_INTEGRATION_URI para los tests que lo solicitan. Sin ellas pueden omitirse; un skip no acredita servicio real. Los casos usan entornos aislados y pueden realizar operaciones de prueba; revisar su base antes de ejecutarlos.

Fallos previsibles y salida honesta

Síntoma	Comprobación	Qué mostrar si no se resuelve a tiempo
App no encuentra modelo	Raíz y RETO1_ARTIFACT_DIR	Ruta y huella del modelo, sin iniciar entrenamiento.
Auditoría falla	Archivo y bytes contra manifiesto	Explicar el fallo; no modificar la huella para ocultarlo.
MongoDB no responde	URI, puerto y ping	Exportación y XML real conservados, identificados como históricos.
Kafka no conecta	Bootstrap y advertised listeners alcanzables	Informe real archivado; no llamar real al mock.
HDFS redirige a host inaccesible	Dirección HTTP del DataNode	Evidencia de recuperación anterior y su fecha.
n8n devuelve 404	Workflow publicado y URL del webhook	Flujo y ficha archivada, sin afirmar un envío actual.
n8n devuelve error	Servicio acciones y confirmación persistido	Mostrar ERROR_N8N y explicar el contrato.
Portal o mapa no carga	Red externa y fuente	HTML archivado o captura, sin repetir agresivamente peticiones.

El plan alternativo consiste en demostrar código y evidencia conservada de forma explícita. No convertir una demostración fallida en un éxito verbal. Las apps pueden usar resultados archivados y aun así ser útiles para explicar; una ejecución en vivo debe identificarse como tal.

Dónde comprobarlo: scripts/transport_rent_api.py · scripts/analyze_rental_sample.py · scripts/analyze_municipal.py · integration_tests/test_bda_sources_live.py · docs/evidencias_cierre_rubrica_2026-11-12/resumen.json.

27 · Qué entregar: paquete y comprobaciones

Los enunciados fijan contenidos; no especifican aquí el nombre final del ZIP, el tamaño máximo del aula virtual ni una fecha de subida. Esos detalles deben ajustarse a la plataforma y a lo que haya indicado el profesorado. No se inventan requisitos de formato que no constan.

Entrega común recomendada

1. Repositorio único accesible, con commit de entrega identificado y README vigente.
2. Código organizado, requirements.txt y uv.lock, lanzador y configuración de ejemplo sin credenciales.
3. Informe actual con arquitectura, tablas, gráficas, conclusiones, límites y procedencia.
4. Datos originales, capturas verificadas, derivados necesarios y manifiestos.
5. Evidencias de HDFS y MongoDB y comandos para reconstruir el recorrido en servicios preparados.
6. Notebooks con salidas conservadas, modelo completo, particiones, MLflow y resultados finales.
7. Workflow n8n y configuración para su demostración local.
8. Presentación y guion de 15 minutos, con material alternativo si falla la red.

El ZIP de esta guía contiene material de estudio y presentación. **No sustituye al repositorio completo ni contiene por sí solo la base MongoDB o los datos de HDFS.** Un archivo de código tampoco equivale a entregar una base de datos en ejecución.

Archivos y carpetas que deben estar accesibles

Entrega	Ubicación vigente
Instrucciones y dependencias	README.md, requirements.txt, requirements-ci.txt, pyproject.toml, uv.lock, .env.example.
Código compartido	src/, scripts/, app/, main.py.
Originales y captura de anuncios/API	data/capturas_oficiales/, data/capturas_anuncios/, data/capturas_api_rentas/.
Resultados territoriales	data/analisis_municipal/ y evidencias de exportación y transporte.
Material ML completo	entrega_ml/data/, models/, resultados/, mlruns/ y notebooks_ejecutados/.
Notebooks fuente	notebooks/ y los bloques académicos organizados.
Automatización MIA	n8n/flujo.json, n8n/compose.yml, scripts/action_server.py.
Evidencia actual	docs/evidencias_entrega_2026-11-12/ y docs/evidencias_cierre_rubrica_2026-11-12/.
Metodología y resultados	docs/METODOLOGIA_ML.md, RESULTADOS_ANUNCIOS.md, CIERRE_RUBRICA.md, DEFENSA.md.
Verificación	tests/, integration_tests/ y .github/workflows/.
Separación académica	BDA + SBD/ y SAA + PIA + MIA/, con src sincronizado al canónico.

Datos en servicios: dos posibilidades que hay que acordar con el docente

Si se evalúa sobre un clúster compartido, indicar host autorizado, base, colecciones y rutas HDFS accesibles. Si se pide una entrega transportable de los servicios, preparar un dump de MongoDB y exportación de archivos HDFS, además de los scripts de reconstrucción. Las evidencias JSON no son un dump de MongoDB. No se afirma que este manual haya generado o restaurado tal dump.

Antes de entregar una copia portable, comprobar en otro directorio o equipo que el modelo carga, las imágenes del README existen, los notebooks conservan salidas, MLflow puede mostrar métricas y el webhook tiene instrucciones. Los servicios Big Data requieren preparación adicional; no se promete un «todo en un comando» para Hadoop, Kafka y MongoDB en cualquier Windows.

Qué excluir del ZIP académico

Entornos .venv, cachés, descargas temporales de Hadoop/JDK, archivos de credenciales, .env reales, logs con datos sensibles y datos personales introducidos durante ensayos. No eliminar originales, modelos, MLflow o informes necesarios por confundirlos con caché. Si el repositorio excede el límite de la plataforma, proporcionar el enlace y los complementos según indique el docente.

Último control

Auditoría correcta; copias sincronizadas; pruebas con resultado registrado; apps abiertas; versión identificada; modelo y test intactos; base y rutas HDFS comprobadas; flujo n8n publicado; presentación y PDF accesibles sin red; nombres de autores completados por el alumnado; enlace y permisos de acceso revisados.

Dónde comprobarlo: README.md · docs/ESTADO_ENTREGA.md · scripts/audit_project.py · scripts/sync_sources.py.

28 · Correspondencia con las cinco asignaturas

Esta tabla sirve para enseñar rápidamente dónde se acredita cada bloque. Es una correspondencia técnica, no una nota garantizada. La evidencia histórica debe leerse con su fecha; el ensayo oral sigue pendiente de realizar por la pareja.

Asignatura y criterio	Evidencia que enseñar
BDA: API y scraping	JSON monetario EMAL, HTML y manifest de anuncios, scripts de captura.
BDA: HDFS	raw/processed, rutas y recuperación idéntica en informes de transporte.
BDA: Kafka e integración	Productor/consumidor, informe de recibidos y persistencia MongoDB.
SBD RA1: integración y modelado	código INE, colecciones, embedding y referencias; carga automatizada.
SBD RA1: limpieza	Cuatro operaciones y rechazo de ausencias/contradicciones sin inventar datos.
SBD RA3: CRUD	Pruebas que comprueban insert, find, update y delete en casos aislados.
SBD RA3: consultas y agregaciones	Cinco consultas útiles, tres pipelines, etapas y resultados.
SBD RA3: índice y consistencia	explain 638/4; incremento atómico de copia y justificación.
SBD RA4: resultados	Indicadores, tres capitales, comparación de 18 municipios y salidas preparadas.
SAA: IA débil y supervisado	Contexto del README y explicación de objetivo continuo.
SAA: dataset/EDA/nulos	Diccionario, cinco gráficas comentadas, retardos e imputación en train.
SAA: cinco familias y MLP	Comparativa y cuatro configuraciones de red con sus motivos.
SAA: métricas y selección	MAE/RMSE/R^2 train/val, baseline, SHAP val y test único.
PIA: documentación y organización	README, estructura, funciones y dependencias.
PIA: pipeline y persistencia	ColumnTransformer, pipeline completo joblib y caso nuevo.
PIA: MLflow y app	Once runs; tabla, captura y tres pestañas de Streamlit.
MIA: negocio y reglas	Problema cuantificado, diez reglas, prioridades y seguridad.
MIA: predicción y acción	Contrafactual, webhook, ocho ramas y ficha persistida.

Aspectos que siguen siendo humanos

La exposición conjunta se corta a los 15 minutos. Después cada alumno responde tres preguntas, una de SAA, una de PIA y una de MIA. El indicador individual pesa 5 %, 15 % y 15 % respectivamente. Los dos deben poder navegar el código y justificar su respuesta.

El nivel máximo por novedad en SAA y por acción distinta de lo visto en clase en MIA depende de todo lo impartido, no solo de los ZIP aportados. No se asegura un diez por tener más archivos, usar SHAP o implementar una ficha. El requisito base y la valoración máxima son cuestiones distintas.

Dónde comprobarlo: docs/CIERRE_RUBRICA.md · docs/CONTRASTE_DOCENTE.md · docs/DEFENSA.md.

29 · Guion de 15 minutos, con reparto y transiciones

Reparto propuesto para ensayar, sin atribuir autoría histórica: persona A explica datos y servicios; persona B explica análisis y ML; A retoma la automatización; ambas cierran. Debe adaptarse a vuestra comprensión real. Ensayar también intercambiando bloques.

Tiempo	Quién	Diapositiva y evidencia	Frase y transición
0:00–1:00	A	1–2: problema y dato Bilbao	«Queremos comparar referencias verificables, no resolver derechos». Pasar a arquitectura.
1:00–2:00	A	3: dos recorridos	«HDFS conserva, Kafka transporta y MongoDB analiza; el modelo usa EMAL».
2:00–3:15	A	4: fuentes y unidades	Abrir un JSON de API y un HTML archivado. «Promoción, anuncio y agregado no son lo mismo».
3:15–4:30	A	5: HDFS/Kafka/MongoDB	Mostrar recuperación e informe 638; explicar persistencia antes del offset.
4:30–5:30	A	6: SBD	Enseñar explain 638/4 y mencionar atomicidad de copia. Ceder a B.
5:30–6:45	B	7: comparación territorial	Vitoria/Bilbao/Donostia con salario regional de 2024; distinguir muestra 2026.
6:45–8:00	B	8: dataset y EDA	601/75/92, lag1, persistencia y preprocesado dentro de train.
8:00–9:30	B	9: modelos y MLP	Nueve regresores; menor MAE y mejora pequeña frente a baseline.
9:30–10:45	B	10: explicación y test	SHAP val; test leído, no repetido; R^2 no es exactitud.
10:45–12:30	B → A	11: caso nuevo	Mostrar periodo, referencia, regla y dos esfuerzos; distinguir prueba 600/1600 del modelo real.
12:30–14:00	A	11: n8n	Consentimiento, ficha y reentrega UUID; «acción local, no concesión».
14:00–15:00	Ambas	12: límites y cierre	Cada persona explica un límite y señala dónde reproducir el proyecto.

Qué llevar abierto

Las dos apps, la presentación, un JSON API, un HTML archivado, resumen de transporte, explain, src/reglas.py y la ficha n8n. MLflow puede estar abierto en otra pestaña, pero no debe consumir minutos buscando experimentos.

Texto de apertura sugerido

«Nuestro proyecto responde a la dificultad de comparar vivienda e ingresos con datos de distinta procedencia. En Bilbao, una renta media de 930,31 euros frente al salario juvenil autonómico de 1.564 supone un escenario del 59,48 %. Recogemos y analizamos referencias verificables y usamos

una predicción municipal para orientar un caso. No concedemos ayudas ni garantizamos disponibilidad».

Texto de cierre sugerido

«Hemos conectado recogida, almacenamiento, eventos, análisis y una orientación explicable. Conservamos qué se obtuvo y cómo se calculó. La muestra no representa todo el mercado y el modelo estima agregados, por lo que la decisión final requiere información individual y revisión humana».

Si vais retrasados

No eliminar la conclusión ni exceder 15 minutos. Resumir la tabla de modelos en ganador, baseline y límite; no enumerar todos los hiperparámetros. Mostrar una consulta y una etapa de pipeline en lugar de leer todo el código. Si n8n no responde tras un intento razonable, enseñar la ficha conservada y explicarlo.

Ensayo recomendado

Primera vuelta: 20 minutos con pausas para dudas. Segunda: 15 minutos sin detener reloj. Tercera: cambiar quién responde preguntas fuera de su bloque. Registrar duración, un fallo observado y una mejora; una práctica oral no se acredita por ejecutar pytest.

Dónde comprobarlo: docs/DEFENSA.md.

30 · Banco de 45 preguntas con respuesta y evidencia

Practicad sin memorizar frases de forma mecánica. La respuesta debe ir acompañada del archivo que la acredita y un límite. Las siguientes preguntas cubren preparación adicional; el enunciado de SAA/PIA/MIA fija tres preguntas individuales por alumno, una de cada asignatura.

1. SAA · ¿Por qué regresión supervisada y no clasificación?

La etiqueta observada es renta media en euros; no una categoría de viabilidad. Se aprende de entradas y objetivos históricos. Enseñar TARGET y FEATURES.

Mostrar: src/ml_data.py.

2. SAA · ¿Por qué IA débil?

Resuelve una tarea específica y no posee razonamiento general ni competencia administrativa. Su cobertura depende del dataset.

Mostrar: README.md.

3. SAA · ¿Qué representa una fila?

Un municipio y trimestre de nuevos contratos. No un joven ni un piso anunciado.

Mostrar: entrega_ml/data/procedencia.json.

4. SAA · ¿Por qué no reparto aleatorio?

Para evaluar periodos posteriores sin mezclar pasado y futuro. Los folds se hacen por trimestre completo.

Mostrar: src/ml_data.py.

5. SAA · ¿Hay fuga al usar retardos en test?

Se predice a un paso con observaciones anteriores; no se usa la renta contemporánea objetivo. Es un backtest revisado y no una previsión autónoma de múltiples pasos.

Mostrar: docs/METODOLOGIA_ML.md.

6. SAA · ¿Por qué no rellenar objetivos ausentes?

Inventar etiquetas cambiaría la verdad que queremos aprender. Se excluyen; las entradas se imputan en train.

Mostrar: src/ml_data.py.

7. SAA · ¿Qué demuestra la dispersión lag1?

Persistencia fuerte, no causalidad. Justifica comparar con la predicción ingenua de repetir la renta anterior.

Mostrar: entrega_ml/resultados/04_dispersion.png.

8. SAA · ¿Por qué MLP_2_capas?

Menor MAE val: 24,20 frente a persistencia 24,63. La mejora es pequeña; test no se usó para elegir.

Mostrar: entrega_ml/models/selection.json.

9. SAA · ¿Por qué KNN tiene train cero?

Con ponderación por distancia puede memorizar; su validación 51,52 muestra que train cero no garantiza generalización.

Mostrar: entrega_ml/resultados/comparativa_mlflow.csv.

10. SAA · ¿Qué cambia en los tres experimentos MLP?

Capacidad 64/32, activación tanh y regularización alpha=0,01 frente a una capa base de 32. Enseñar la factoría y resultados.

Mostrar: src/ml_models.py.

11. SAA · ¿MAE 28,67 significa un error máximo de 28,67?

No. Es una media de errores absolutos; puede haber errores mayores y no es un intervalo predictivo.

Mostrar: docs/TEST_FINAL_EMAL.json.

12. SAA · ¿R^2 0,9189 es 91,89 % de aciertos?

No. Es una comparación de suma de errores cuadrados con la referencia de predecir la media.

Mostrar: src/ml_models.py.

13. SAA · ¿SHAP demuestra causalidad?

No. Aproxima contribuciones del modelo en 30 filas de validación frente a fondo de train; magnitud absoluta no es efecto causal.

Mostrar: entrega_ml/resultados/shap_metadatos.json.

14. SAA · ¿SMOTE mejoró el modelo?

En la clasificación auxiliar F1 pasó de 0,8254 a 0,8167. No mejora esa métrica y no se aplica a la renta continua.

Mostrar: entrega_ml/resultados/comparacion_smote.json.

15. PIA · ¿Qué contiene el joblib?

Pipeline con transformaciones ajustadas y regresor; un caso nuevo recibe el mismo preprocesado.

Mostrar: src/ml_workflow.py.

16. PIA · ¿Por qué StandardScaler y OneHotEncoder?

Escala numéricas para modelos sensibles a distancias y convierte identificadores categóricos sin orden artificial. Se ajustan en train.

Mostrar: src/ml_workflow.py.

17. PIA · ¿Cómo vuelve la predicción a euros?

TransformedTargetRegressor invierte la transformación de la variable objetivo.

Mostrar: src/ml_workflow.py.

18. PIA · ¿Dónde están los experimentos MLflow?

Metadatos en SQLite y artefactos en entrega_ml/mlruns; tabla extraída de las ejecuciones, no números inventados.

Mostrar: src/mlflow_tracker.py.

19. PIA · ¿Por qué once runs y nueve modelos en tabla?

Nueve de regresión y dos auxiliares con/sin SMOTE. No mezclar métricas de tareas distintas.

Mostrar: entrega_ml/resultados/comparacion_smote.json.

20. PIA · ¿Abrir Streamlit entrena?

No; carga el pipeline conservado y lee el test ya sellado. La pestaña mantiene un nombre histórico.

Mostrar: app/streamlit_app.py.

21. PIA · ¿Cómo se evita repetir test?

evaluate_test_once reutiliza el informe y verifica huellas. Modificar modelo o test no valida un resultado nuevo automáticamente.

Mostrar: src/ml_workflow.py.

22. PIA · ¿Cómo se mantienen copias académicas?

src es la fuente canónica y sync_sources verifica o sincroniza las copias; no se editan versiones independientes a mano.

Mostrar: scripts/sync_sources.py.

23. PIA · ¿Qué prueba un mock?

La lógica aislada; no planes explain ni transporte real. Los servicios tienen evidencias e integraciones separadas.

Mostrar: src/database/indexes.py.

24. MIA · ¿Cómo afecta la predicción a la decisión?

R_REFERENCIA compara referencia e ingreso. El contrafactual 600/1600 cambia decisión con el mismo contrato.

Mostrar: tests/test_business_rules.py.

25. MIA · ¿Por qué dos esfuerzos?

Uno usa contrato y deudas; el otro el escenario de pagar la referencia municipal. No se reemplaza un dato declarado por una media.

Mostrar: src/reglas.py.

26. MIA · ¿Qué pasa si varias reglas coinciden?

Se ordenan por prioridad y gana la primera aplicable. Seguridad y requisitos preceden a recomendaciones generales.

Mostrar: src/reglas.py.

27. MIA · ¿Por qué diez reglas y ocho ramas?

Varias reglas comparten decisión; el flujo se ramifica por decisión, no por regla.

Mostrar: n8n/flujo.json.

28. MIA · ¿El 40 % es ley?

No. Es un umbral interno de orientación. Las condiciones normativas tienen otra fuente y pasan a revisión cuando hay excepciones.

Mostrar: src/reglas.py.

29. MIA · ¿Qué hace la acción?

Persiste una ficha local distinta según decisión. No envía correo ni concede ayudas; cita_prioritaria no reserva agenda.

Mostrar: scripts/action_server.py.

30. MIA · ¿Qué datos salen del formulario?

UUID, decisión, regla, referencia, origen y versión. No salario, nombre o perfil completo; se exige consentimiento.

Mostrar: src/n8n.py.

31. MIA · ¿Cómo evita duplicados?

Mismo UUID y contenido recupera acción existente. Un UUID reutilizado con distinto contenido se rechaza.

Mostrar: scripts/action_server.py.

32. MIA · ¿Y si cambia la norma?

La regla del año envía a revisión fuera de 2026. Mantener normativa requiere actualización revisada; el modelo no aprende leyes.

Mostrar: src/reglas.py.

33. BDA/SBD · ¿Por qué API y scraping?

Son dos vías exigidas: JSON estructurado oficial y HTML de anuncios. Tienen unidades y contratos distintos.

Mostrar: scripts/capture_emal_api.py.

34. BDA/SBD · ¿Una promoción equivale a un piso libre?

No; es un registro de catálogo. Por eso se distingue tipo_registro y se excluye de precios de anuncios.

Mostrar: docs/RESULTADOS_ANUNCIOS.md.

35. BDA/SBD · ¿Para qué HDFS si ya hay MongoDB?

Conserva archivos originales y procesados recuperables. MongoDB guarda documentos preparados para consultar.

Mostrar: src/hdfs/client.py.

36. BDA/SBD · ¿Por qué commit de Kafka después de MongoDB?

Para no marcar como procesado un evento no persistido. La reentrega se maneja con upsert idempotente.

Mostrar: src/kafka/consumer.py.

37. BDA/SBD · ¿Es exactamente una vez entre sistemas?

No se demuestra una transacción Kafka-MongoDB; se demuestra reentrega con persistencia idempotente.

Mostrar: scripts/run_pipeline.py.

38. BDA/SBD · ¿Qué implica una consulta vacía?

Puede reflejar ausencia de un dato publicado. No se inventan ascensores o programas para obtener resultados.

Mostrar: src/database/queries.py.

39. BDA/SBD · ¿Qué prueba el índice?

En esa consulta reduce examinados de 638 a 4. Ambos tiempos son 0 ms, por lo que no cuantifica aceleración.

Mostrar: docs/evidencias_entrega_2026-11-12/anuncios/explain.json.

40. BDA/SBD · ¿Por qué sin transacción multidocumento?

El contador y fecha cambian atómicamente en un documento; no requiere actualizar municipios de forma inseparable.

Mostrar: src/database/consistency.py.

41. BDA/SBD · ¿Cuántos municipios tienen salario medido?

La comparación usa un único salario autonómico de 2024; no se han medido salarios municipales en esta entrega.

Mostrar: src/municipal_analysis.py.

42. BDA/SBD · ¿Qué garantiza SHA-256?

Igualdad de bytes respecto al manifiesto; no verdad, actualidad ni licencia de los datos.

Mostrar: scripts/audit_project.py.

43. BDA/SBD · ¿Se pueden generalizar 17 anuncios?

No. Son una muestra acotada de conveniencia de dos portales, con modalidades y posible duplicación física.

Mostrar: docs/RESULTADOS_ANUNCIOS.md.

44. Todas · ¿Qué es distinto de clase?

Clientes Kafka/HDFS y biblioteca de MLP. La integración es más amplia; la novedad se valora contra todo el curso, no solo los ZIP.

Mostrar: docs/CONTRASTE_DOCENTE.md.

45. Todas · ¿Cuál es la mejora futura prioritaria?

Mejor cobertura temporal y territorial, evaluación prospectiva y datos de ingresos adecuados. Son propuestas, no trabajo ya implementado.

Mostrar: docs/METODOLOGIA_ML.md.

31 · Límites, errores de discurso y próximos pasos

Frases que hay que sustituir

Evitar	Formulación defendible
«Tenemos 638 viviendas disponibles»	«Transportamos 621 registros de promociones y 17 identidades de anuncios».
«El modelo dice si una persona es viable»	«Predice una referencia municipal; las reglas orientan un caso declarado».
«El sueldo de Bilbao es 1.564 €»	«Usamos un salario juvenil autonómico de 2024 como escenario común».
«El 59,48 % de los jóvenes no puede emanciparse»	«La renta de referencia supone el 59,48 % de ese salario común».
«R^2 es porcentaje de acierto»	«Mide mejora cuadrática respecto a predecir la media del conjunto».
«El error siempre es menor de 28,67 €»	«Ese es el error absoluto medio de las 92 observaciones del test».
«SHAP demuestra las causas»	«Describe contribuciones del modelo en una muestra de validación».
«SMOTE mejoró el modelo de renta»	«En la tarea auxiliar redujo F1; no se aplica al objetivo continuo».
«n8n envía ayudas o reserva citas»	«Guarda una ficha local; la atención y resolución requieren revisión humana».
«El índice fue mucho más rápido»	«Examinó cuatro documentos frente a 638; no medimos una diferencia temporal».
«Todo se ejecuta en vivo al abrir el visor»	«El visor carga exportaciones verificadas; el transporte real tiene su evidencia separada».
«TimeSeriesSplit es una novedad no vista»	«El PDF de clase ya contempla validación temporal».

Límites que sí conviene mencionar

Muestra de anuncios pequeña y no aleatoria; precios pedidos distintos de contratos; cobertura municipal limitada; salarios regionales; revisión histórica de EMAL; predicción a un paso y mejora pequeña; ausencia de intervalos calibrados; reglas normativas parametrizadas; acciones locales; dependencias externas de red y cartografía. Son límites concretos observados, no una lista hipotética de riesgos.

Propuestas futuras, claramente separadas

Ampliar y diversificar capturas respetando condiciones; evaluar con nuevos periodos no utilizados para elegir; disponer de fuentes de ingreso territorial comparables; revisar normativa con especialistas; estudiar incertidumbre y equidad de cobertura; mejorar despliegue reproducible de servicios y accesibilidad de la interfaz. No se anuncian como funcionalidades implementadas ni se ejecutan para modificar el modelo cerrado.

Lista personal para los dos

Cada uno debe poder dibujar la arquitectura, identificar una fila y sus unidades, explicar los cinco EDA, describir cada familia y los cuatro MLP, interpretar MAE/RMSE/R^2, enseñar un pipeline MongoDB, justificar el índice y la atomicidad, seguir una regla hasta la ficha y arrancar las apps sin entrenar.

Si una respuesta no se conoce, reconocerlo y localizar el archivo es mejor que atribuir al sistema algo que no hace. La defensa será más sólida si las afirmaciones coinciden con el código y la evidencia que tenéis delante.

Dónde comprobarlo: docs/CIERRE_RUBRICA.md · docs/RESULTADOS_ANUNCIOS.md · docs/METODOLOGIA_ML.md.

32 · Glosario y mapa de fuentes

Término	Significado en el proyecto
API	Interfaz para solicitar o intercambiar datos con un contrato definido.
Scraping	Extracción de campos desde HTML.
Raw / processed	Original conservado / derivado preparado.
SHA-256	Huella de los bytes para comprobar integridad.
INE	Código territorial estable utilizado para unir municipios.
HDFS	Sistema de archivos distribuido de Hadoop.
NameNode / DataNode	Metadatos del sistema de archivos / almacenamiento de bloques.
Tópico / partición / offset	Flujo Kafka / subdivisión / posición en esa partición.
Upsert	Actualizar si existe, insertar si no existe.
Idempotencia	Repetir la misma operación no crea un efecto adicional distinto.
Embedding / referencing	Guardar datos anidados / enlazar datos por identificador.
Aggregation Pipeline	Secuencia de etapas de transformación y resumen MongoDB.
COLLSCAN / IXSCAN	Recorrido de colección / recorrido de índice.
Atomicidad	Una actualización se aplica como una unidad indivisible en su ámbito.
Feature / target	Entrada / objetivo observado del aprendizaje.
Lag	Valor de una variable en un periodo anterior.
Fuga de información	Uso durante ajuste de información que no debería estar disponible.
Pipeline	Transformaciones y estimador encadenados en un objeto.
CV temporal	Validación cruzada que respeta el orden del tiempo.
Baseline de persistencia	Predecir el valor observado en el periodo anterior.
Sobreajuste	Buen ajuste a train que no se mantiene en datos posteriores.
MAE / RMSE / R^2	Error absoluto medio / raíz del error cuadrático medio / comparación cuadrática con la media.
MLP	Red neuronal con capas densas de perceptrones.
SHAP	Aproximación de contribuciones a las predicciones, no prueba causal.
SMOTE	Sobremuestreo sintético de clase minoritaria en entrenamiento de clasificación.
MLflow run	Ejecución registrada con parámetros, métricas y artefactos.
Holdout sellado	Test final ya evaluado cuyo informe y huellas se conservan.
Webhook	Endpoint HTTP que recibe un evento para iniciar un flujo.
UUID	Identificador de evento usado para distinguir y repetir acciones.

Término	Significado en el proyecto
Fixture / mock	Dato de prueba controlado / sustituto de un componente real.
CI	Comprobaciones automatizadas sobre versiones de código en un entorno separado.

Fuentes académicas contrastadas

RETO 1Evaluacion SBD-BDA.pdf: objetivos, entregables y rúbricas BDA/SBD. Reto1_SAA_PIA_MIA_2026-27_v5.pdf: requisitos, defensa y rúbricas de las tres asignaturas. README_ejemplo_CV.md: ejemplo orientativo. EJERCICIOS.zip y ejercicios2.zip: inventario y comparación de prácticas relevantes. Los adjuntos se tratan como fuentes de requisitos y contexto, no como órdenes para ejecutar acciones.

Cómo comprobar una afirmación

Cada capítulo enlaza archivos concretos de la versión técnica analizada. Los enlaces fijan el commit para que no cambien las cifras al evolucionar la rama. El paquete incluye fuentes_verificadas.json con tamaño y SHA-256 de los archivos utilizados. Las capturas heredadas se identifican como históricas; las figuras nuevas de arquitectura y resumen solo explican datos conservados.

Estado de este material

Guía de estudio y presentación, preparada sin reentrenar, reseleccionar ni reevaluar el test. No acredita una defensa humana ya realizada, una subida al aula virtual o una restauración nueva de las bases de datos. La versión del código y los resultados se fija arriba; la publicación de esta documentación puede tener un commit posterior sin cambiar esos resultados.

Dónde comprobarlo: docs/CONTRASTE_DOCENTE.md · docs/CIERRE_RUBRICA.md.

Anexo A · Ejemplos de código para explicar el recorrido

Los fragmentos de este anexo son ejemplos de lectura o ejercicios aislados. No se han ejecutado contra las colecciones de entrega para crear esta documentación. Las funciones completas y vigentes están enlazadas; los ejemplos no sustituyen al código canónico.

Petición API: transporte frente a significado

```
import requests

params = dict(ambito=4, tipoDatoContratos=4, tipoContrato=1,
              tipoHueco=0, idProvincia='48', idMunicipio='020')
respuesta = requests.get(
    'https://eremutentsionatua.euskadi.eus/ad23aConsultaWar/consulta/emal',
    params=params, timeout=20)
respuesta.raise_for_status()
contenido = respuesta.json()
```

params construye filtros y URL; timeout limita la espera de red; raise_for_status comprueba HTTP; json interpreta el cuerpo. Ninguna de esas líneas garantiza todavía que la respuesta contenga el trimestre y tipo de renta esperados. El capturador real añade archivo original, procedencia, validación semántica y huellas. Para la defensa se muestra la captura conservada, sin necesidad de repetir la petición.

CRUD sin tocar la entrega

Usar solo una base de ejercicio aislada y comprobar que su nombre no coincide con una base de resultados. Este ejemplo inventado crea y elimina únicamente su documento identificado como ejemplo_manual; no es una vivienda real.

```
from pymongo import MongoClient

cliente = MongoClient('mongodb://127.0.0.1:27017')
coleccion = cliente['reto1_ejemplos_manual']['ejemplos_crud']
clave = {'_id': 'ejemplo_manual'}

coleccion.update_one(clave, {'$set': {
    'es_sintetico': True, 'precio_ejemplo': 500}}, upsert=True)
assert coleccion.find_one(clave)['precio_ejemplo'] == 500
coleccion.update_one(clave, {'$set': {'precio_ejemplo': 550}})
assert coleccion.find_one(clave)['precio_ejemplo'] == 550
resultado = coleccion.delete_one(clave)
assert resultado.deleted_count == 1
assert coleccion.find_one(clave) is None
cliente.close()
```

update_one con upsert acredita inserción/actualización, find_one consulta y delete_one borra el ejemplo. assert hace explícita la comprobación esperada. En el proyecto las identidades de vivienda usan fuente e id_origen, con índices y validación; aquí se simplifica a un _id fijo para explicar CRUD.

Consulta útil de solo lectura

```
filtro = {
    'es_sintetico': False,
    'tipo_registro': {'$ne': 'promocion'},
    'tipo_operacion': 'alquiler',
    'municipio': 'Bilbao',
    'precio': {'$type': 'number', '$gt': 0, '$lte': 2000},
    'superficie_m2': {'$gte': 55}
}
proyeccion = {'_id': 0, 'titulo': 1, 'precio': 1, 'superficie_m2': 1}
filas = list(db.viviendas.find(filtro, proyeccion).sort('precio', 1))
```

Las claves simultáneas combinan condiciones. $ne excluye promociones, $type evita tratar textos como precio y $gte selecciona superficie mínima. La proyección reduce campos devueltos y sort ordena ascendente. Los cuatro resultados de evidencia corresponden a la muestra y parámetros conservados; otra base puede devolver otro número.

Pipeline: de documentos a resumen

```
pipeline = [
    {'$match': {
        'es_sintetico': False,
        'tipo_registro': {'$ne': 'promocion'},
        'tipo_operacion': 'alquiler',
        'precio': {'$type': 'number', '$gt': 0}}},
    {'$group': {
        '_id': '$codigo_municipio',
        'anuncios': {'$sum': 1},
        'precio_medio': {'$avg': '$precio'}}},
    {'$sort': {'precio_medio': 1}}
]
resumen = list(db.viviendas.aggregate(pipeline))
```

El signo $ delante de codigo_municipio significa leer el valor del campo, no agrupar todo por el texto literal. $sum:1 cuenta documentos; $avg resume precios publicados. No se deduce asequibilidad personal. El pipeline real añade validación de código, proyección, cruces y otras salidas según la pregunta.

Construcción del pipeline ML, sin ejecutarlo aquí

```
from sklearn.pipeline import Pipeline
from sklearn.compose import TransformedTargetRegressor
from sklearn.preprocessing import StandardScaler
from src.pipeline import build_preprocessor
from src.ml_data import NUMERIC, CATEGORICAL
from src.ml_models import regression_models

estimador = regression_models()['MLP_2_capas']
pipeline = Pipeline([
    ('preprocesado', build_preprocessor(NUMERIC, CATEGORICAL)),
    ('modelo', TransformedTargetRegressor(
        regressor=estimador, transformer=StandardScaler()))
])
```

Construir el objeto no lo ajusta. El entrenamiento histórico llamó fit solo con train y evaluó validación antes de elegir. En la defensa se carga el pipeline ya ajustado desde best_model.joblib y se llama predict únicamente para un caso nuevo; no se usa este ejemplo para volver a entrenar o reevaluar test.

Leer un evento n8n

Un payload de ejemplo contiene event_id, decision, regla_id, renta_estimada, origen y version. La decisión determina la rama y el identificador permite detectar repeticiones. El receptor añade persistido y los datos de la ficha. Guardar primero el documento y comprobar la confirmación explica por qué la app puede distinguir un envío efectivo de una respuesta incompleta.

Dónde comprobarlo: scripts/capture_emal_api.py · src/database/data_loader.py · src/database/queries.py · src/database/aggregations.py · src/pipeline.py · src/ml_workflow.py · src/n8n.py.

Anexo B · Organización, Git y evidencia automatizada

Fuente canónica y copias de asignaturas

src/ contiene las implementaciones compartidas. Los bloques académicos mantienen copias sincronizadas para su organización de entrega. scripts/sync_sources.py --check compara sin modificarlas. Si se cambia una implementación en el futuro, se sincroniza antes de entregar; editar una copia por separado crea divergencia y puede hacer que dos lanzadores muestren comportamientos distintos.

main.py es el portal de entrada. Las opciones --audit, --test y --pdf lanzan comprobaciones o generación de informe; --subject selecciona bloque y --visual abre el visor BDA. --pipeline es una acción del recorrido BDA y no una orden de entrenar el modelo IA. El entrenamiento está separado en un script explícito y no forma parte del arranque de la entrega cerrada.

Ramas y trabajo de pareja

Las ramas personales se configuran localmente con git config dev.branch; no se impone una identidad al compañero en el código compartido. test integra trabajo validado y main conserva la versión estable. Los commits permiten revisar cambios y autoría registrada; una propuesta de reparto en las diapositivas no modifica esa autoría.

El flujo de AGENTS.md exige revisar cambios, comprobar que no se añaden secretos, ejecutar pruebas, publicar la rama personal, integrar y validar test antes de main y volver a la rama personal. No se necesitan force push ni reset destructivo para preparar documentación.

Qué valida GitHub Actions

Los workflows separan comprobaciones de código, fuentes, MongoDB, transporte y n8n. Algunos levantan servicios temporales reales y guardan resultados. Un estado verde acredita los pasos de esa ejecución, versión y entorno; no prueba disponibilidad permanente de los portales ni una defensa oral.

Tipo de evidencia	Qué acredita	Qué no acredita
pytest unitario con fixture/mock	Contratos y lógica controlada	Servicios distribuidos activos.
Integración real	Operaciones con servicios identificados	Disponibilidad continua o producción segura.
XML JUnit	Casos, fallos, omisiones y duración registrados	Validez de toda conclusión de negocio.
Captura del navegador	Pantalla observada de una versión y caso	Todos los caminos de la aplicación.
Manifiesto SHA-256	Integridad de archivos concretos	Exactitud sustantiva o actualidad de la fuente.
Run remoto de CI	Pasos de una revisión en entorno separado	Éxito de una ejecución local posterior.

Cierre y futuras modificaciones

La documentación puede evolucionar sin cambiar el modelo. Antes y después de una modificación, auditar las seis capturas oficiales, los 88 artefactos ML, el test sellado, las copias académicas y las capturas de anuncios/API. Las actualizaciones futuras del modelo requieren un procedimiento nuevo de evaluación; el test ya observado no vuelve a ser una reserva limpia.

La entrega incluye informes PDF generales en la raíz y en los bloques académicos. Esta guía añade material pedagógico: no sustituye las evidencias originales ni introduce números nuevos de entrenamiento. La validación de esta tarea se guarda separada de la del cierre anterior.

Dónde comprobarlo: AGENTS.md · main.py · scripts/sync_sources.py · scripts/git_workflow.py · .github/workflows/bda-candidate.yml · .github/workflows/mongodb-real.yml · .github/workflows/n8n-real.yml.